

MASTERARBEIT
Computer Vision Group

Electrical Engineering and Computer Science

A Computer Vision Pipeline for Football Table
Analytics and Game Insights

Aakash Deshpande

Matriculation Number: 1701953

Study Program: Mechatronics, Master of Science (M.Sc)

Supervisor I : Prof. Dr. Michael Möller

Supervisor II: Mr. Alexander Auras

March, 2026

Eigenständigkeitserklärung

Ich versichere, dass ich die schriftliche Ausarbeitung selbständig angefertigt und keine anderen als die angegebenen Hilfsmittel benutzt habe. Alle Stellen, die dem Wortlaut oder dem Sinn nach (inkl. Übersetzungen) anderen Werken entnommen sind, habe ich in jedem einzelnen Fall unter genauer Angabe der Quelle (einschließlich des World Wide Web sowie generativer KI und anderer elektronischer Datensammlungen) deutlich als Entlehnung kenntlich gemacht. Dies gilt auch für angefügte Zeichnungen, bildliche Darstellungen, Skizzen und dergleichen. Ich nehme zur Kenntnis, dass die nachgewiesene Unterlassung der Herkunftsangabe als versuchte Täuschung gewertet wird.

Ich bestätige, dass die elektronische Version inhaltlich mit der gedruckten Version übereinstimmt.

30.03.2026

Datum



Unterschrift

Declaration of Authorship

I certify that I have prepared this written work independently and have not used any aids other than those specified. I have clearly identified all passages that are taken from other works in terms of wording or meaning (including translations) as borrowed material in each individual case, stating the exact source (including the World Wide Web as well as generative AI and other electronic data collections). This also applies to attached drawings, pictorial representations, sketches and the like. I acknowledge that the proven omission of the indication of origin will be regarded as attempted deception.

I confirm that the content of the electronic version is the same as the printed version.

Abstract

Automated game analysis for table football, commonly known as foosball or Tischkicker, presents a well-known challenge in the field of computer vision and machine learning. In this work, a pipeline is built to predict the winner of a foosball game given an MP4 video recorded by a single fixed overhead webcam as input. To achieve this, the camera is first calibrated using Zhang’s method to recover the intrinsic parameters and remove lens distortion. ArUco marker-based pose estimation computes a homography that corrects the perspective distortion and generates a top-down bird’s-eye view of the playing field. The ball is detected using HSV colour thresholds and tracked using a Kalman filter that maintains the ball trajectory even during occlusion. The positions of all 22 player figurines across eight rods are estimated using intensity profiles sampled along each rod. Additionally, ball-player contact events are detected and classified using an entry-exit zone state machine. Finally, a Temporal Convolutional Network (TCN) with dilated causal convolutions, batch normalisation, and dropout is trained on sequences of 25 positional and temporal features to predict the winner. Each video is split into six non-overlapping sequences and a hard and soft voting strategy combines the sequence predictions into a final video-level winner prediction.

The hit detection engine detects 3184 hits across 41 non-Draw videos with Dribble and Kick as the most frequent categories. The TCN achieves a mean sequence-level accuracy of 0.487 ± 0.017 , a hard-vote video-level accuracy of 0.575 ± 0.100 , and a soft-vote video-level accuracy of 0.525 ± 0.050 across five random seeds, consistently above both the uniform random baseline of 0.333 and the majority-class baseline of 0.375. The results demonstrate that positional features extracted from a single overhead webcam carry meaningful information about the foosball game outcome.

Acknowledgment

I wish to convey my sincere gratitude to Professor Dr. Michael Möller, University of Siegen for giving me the opportunity to work on this project and for diligently ensuring that all administrative details were handled seamlessly.

I would like to express my sincere gratitude to Alexander Aurus of the University of Siegen, my supervisor, for his invaluable time, constant guidance, and steadfast support throughout this work. His guidance was crucial for the organised and timely completion of the project. His consistent support kept me on course to reach my objectives. The regular weekly meetings ensured that I received constant feedback on my work.

I would also like to acknowledge the students of the university who were working on the same setup with different goals. Their initiative, enthusiasm, and willingness to implement ideas ahead of schedule encouraged me to stay on course.

Additionally, I would like to thank the staff of Computer vision group for their approachability and readiness to help whenever needed.

Finally, I would like to thank my friends and AI enthusiasts Amey Joshi, Jahnvi Dulala, Lasya Indukuri, Kunal, and Nikhil for generously volunteering their time. Their assistance and participation made sure I had the data for initial implementation of the project. This project would not have been possible without such data within a short timeframe.

Contents

Abstract	II
Acknowledgment	III
Table of Contents	VII
List of Figures	IX
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Contributions	1
1.4 Related Work	2
1.5 Thesis Structure	3
2 Background	4
2.1 Camera Model and Lens Distortion	4
2.1.1 The Pinhole Camera Model and The Intrinsic Matrix	4
2.1.2 Lens Distortion	5
2.2 Camera Calibration	6
2.2.1 Zhang’s Calibration Method	6
2.2.2 Corner Detection and Sub-pixel Refinement	7
2.2.3 Reprojection Error	8
2.2.4 Undistortion Maps	8
2.3 Pose Estimation and Perspective-n-Point Problem	8
2.3.1 ArUco Fiducial Markers	8
2.3.2 The Perspective-n-Point Problem and Levenberg–Marquardt Refinement	9
2.4 Homography and Perspective Correction	11
2.4.1 Projective Transformations and Homography	11
2.4.2 Homography Estimation and Perspective Warping	11
2.5 Color Spaces and HSV-Based Detection	12
2.5.1 BGR vs HSV Color Representation	12
2.5.2 HSV Thresholding for Ball Detection	13
2.5.3 Morphological Cleaning	13

2.6	Contour Detection and Blob Fitting	14
2.6.1	Contour Extraction	14
2.6.2	Minimum Enclosing Circle	15
2.7	Kalman Filter for Object Tracking	15
2.7.1	State-Space Representation	15
2.7.2	Prediction and Correction Steps	16
2.7.3	Tracking Under Occlusion	17
2.8	1D Signal Processing for Rod Detection	17
2.8.1	The Bilateral Filter	17
2.9	Temporal Convolutional Networks	18
2.9.1	Sequence Classification	18
2.9.2	Dilated Causal Convolutions	19
2.9.3	Residual Connections	20
2.9.4	Batch Normalisation and Dropout	20
2.9.5	Global Average Pooling	21
2.9.6	Class Imbalance and Weighted Loss	22
2.10	Voting for Video-Level Prediction	23
2.10.1	Hard Voting and Soft Voting	23
2.11	Multi-Seed Evaluation	23
3	Method	25
3.1	System Overview	25
3.1.1	Hardware Setup	25
3.1.2	Pipeline Architecture	27
3.1.3	Image Representations Used Throughout the Pipeline	27
3.2	Camera Calibration	28
3.2.1	Calibration Video Recording	28
3.2.2	Calibration Image Extraction and Quality Filtering	28
3.2.3	Calibration Solve and Iterative Refinement	29
3.2.4	Calibration Validation	29
3.3	Data Recording	30
3.3.1	Dataset Construction and Labeling	30
3.3.2	Train/Test Split Design	30
3.4	Perspective Correction	30
3.4.1	3D World Coordinate System Definition	30
3.4.2	ArUco Marker Detection and Anchor Point Extraction	31
3.4.3	Pose Estimation and Homography Computation	32
3.5	Ball Color Calibration	33
3.5.1	Motivation for Per-Setup Calibration	33

3.5.2	HSV Range Computation	33
3.6	Ball Detection and Tracking	34
3.6.1	HSV Masking and Morphological Cleaning	34
3.6.2	Kalman Filter for Robust Tracking	34
3.6.3	Ball Status State Machine	35
3.7	Player Position Detection	35
3.7.1	Rod Line Projection into the Warped Image	35
3.7.2	Intensity Profile Extraction and Filtering	36
3.7.3	Peak Detection, Region Merging, and Stopper Identification	36
3.7.4	Smart Stabilisation	37
3.8	Hit Detection — Branch 1	37
3.8.1	Hit Zone Definition and Ball-in-Zone Check	37
3.8.2	Entry/Exit State Machine Per Player	38
3.8.3	Hit Classification Logic	39
3.9	Feature Extraction — Branch 2	41
3.9.1	Intentional Simplification Relative to Branch 1	41
3.9.2	The 25 Feature Columns	41
3.10	Post-Processing and Dataset Construction	41
3.10.1	Frame Filtering	41
3.10.2	Temporal Sequencing and Local Frame Norm	42
3.10.3	Zero-Padding and Final Dataset Arrays	42
3.11	TCN Model Architecture and Training	43
3.11.1	Input Representation and Dataset Class	43
3.11.2	TCN Block Architecture	43
3.11.3	Training Procedure and History Logging	44
4	Experiments	46
4.1	Implementation Details	46
4.1.1	Experimental Environment and Model Hyperparameters	46
4.2	Calibration Results	46
4.2.1	Corner Detection Quality	46
4.2.2	Reprojection Error and Intrinsic Parameters	47
4.2.3	Undistortion Quality	48
4.2.4	Dataset Statistics	50
4.3	Perspective Correction Results	51
4.3.1	Homography Quality and Bird’s-Eye View	51
4.3.2	Rod Line Projection Accuracy	52
4.4	Ball Detection and Tracking Results	53
4.4.1	HSV Masking Result	53

4.4.2	Detection Rate and Status Distribution	54
4.5	Player Position Detection Results	55
4.5.1	Intensity Profile and Color Strip	55
4.5.2	Peak Detection and Player Position	56
4.5.3	Final Player Position Overview	58
4.6	Hit Detection Results	58
4.6.1	Hit Count and Type Distribution	58
4.6.2	Hit Detection Qualitative Results	59
4.7	TCN Classification Results	61
4.7.1	Evaluation Protocol	61
4.7.2	Training Curves and Convergence Behaviour	61
4.7.3	Sequence-Level Accuracy Across Seeds	62
4.7.4	Confusion Matrix Analysis	63
4.7.5	Predicted Probability Distributions	64
4.7.6	Video-Level Accuracy via Voting	65
5	Conclusion	67
5.1	Summary and Connection to the Introduction	67
5.2	Limitations and Future Work	67
	Bibliography	69

List of Figures

2.1	Pinhole camera model	5
2.2	Illustration of barrel and pincushion distortion	6
2.3	ArUco markers of different sizes	9
2.4	Illustration of the Perspective-n-Point problem	10
2.5	Example of perspective correction using homography	12
2.6	The HSV color space represented as a cone	13
2.7	Illustration of morphological opening and closing operations on a binary image .	14
2.8	Illustration of dilated causal convolutions	19
2.9	Residual connection in a TCN block	20
2.10	Dropout during training and inference	21
2.11	Comparison between flattening and global average pooling	22
2.12	Illustration of Hard voting and soft voting	23
3.1	Red ball used in recordings	25
3.2	PLUSONIC USB webcam	26
3.3	Complete experimental setup	26
3.4	Four ArUco markers used	32
3.5	Hit zone geometry derivation	38
3.6	TCN architecture overview and block detail	44
4.1	Detected checkerboard corners	47
4.2	Original distorted calibration frame	49
4.3	Undistorted calibration frame	50
4.4	ArUco anchor and field corner points	51
4.5	Warped bird’s-eye view of playing field	52
4.6	Projected rod lines on warped image	53
4.7	HSV binary mask after thresholding	54
4.8	Colour strips along all 8 rods	55
4.9	Raw grayscale intensity profiles	56
4.10	Peak detection and player positions	57
4.11	Player position overview with hit zones	58
4.12	Warped view during active hit event	60
4.13	Real-time information panel at frame 106	60

4.14	Training curves for seed 21	62
4.15	Per-seed accuracy by class	63
4.16	Mean and std accuracy across seeds	63
4.17	Confusion matrix for seed 21	64
4.18	Predicted probability distributions seed 21	65
4.19	Video-level voting results seed 21	66
4.20	Final accuracy comparison across evaluation levels	66

1 Introduction

1.1 Motivation

Automated game analysis using computer vision has become increasingly popular among researchers in the recent years. Regular webcams, image processing methods and machine learning algorithms can be used to automate sports analysis by extracting meaningful information from a video stream at a very low cost, making it possible to automate many analysis tasks that previously required manual observation or expensive dedicated hardware.

Table football, commonly known as foosball or Tischkicker, presents a particularly interesting setting for automated game analysis. The game is played on a compact table in a controlled environment and can be fully observed from a single fixed overhead camera and no expensive multi-camera setup or specialised sensors are required. Despite this simplicity of setup, the game itself is fast and rich in events. The ball moves at high speeds of 10 m/s and is frequently occluded by rods and figurines. Meaningful events such as hits, kicks, and goals occur at the timescale of milliseconds. There are 22 player figurines across eight rods whose positions change continuously, and each figurine can interact with the ball in different ways depending on how the rod is moved. This combination of a controlled observable environment and genuinely challenging fast dynamics makes foosball an interesting problem for automated game analysis using computer vision.

1.2 Problem Statement

Given a recorded MP4 video of a foosball game captured by a single fixed overhead webcam, the goal is to predict the winner of the game i.e. White, Black, or Draw without any manual intervention, additional sensors, or specialised hardware.

1.3 Contributions

This thesis makes the following contributions:

1. **A complete end-to-end pipeline from raw video to winner prediction.** The

pipeline is designed around a branched architecture in which a shared trunk handles camera calibration, perspective correction, ball detection and tracking, and player position detection. Two independent branches serve different downstream purposes. Branch 1 handles hit detection and analysis, while Branch 2 handles feature extraction and sequence classification.

2. A hit detection and classification system. An entry-exit zone state machine is developed that detects ball-player contact events. Each of the 22 player figurines has a hit zone centred at the player position. The entry-exit state machine logic classifies each contact based on entry velocity, contact duration, and exit velocity. This logic classifies the hit into one of five categories: Kick, Dribble, Block, Brush, and Stop.

3. An intensity-profile-based player position detection method. Rather than using complex methods to detect the figurines on the rod, a simple yet effective method is used to detect the player positions with the help of intensity profiles and bilateral filtering. This system is robust to slight varying lighting conditions and runs smoothly on a CPU.

4. A TCN-based sequence classifier for game outcome prediction. A Temporal Convolutional Network [1] is trained on sequences of 25 features extracted from game videos. The model is evaluated across five random seeds using a multi-seed evaluation protocol with both sequence-level and video-level voting accuracy reported. The model achieves a mean sequence-level accuracy of 0.487 ± 0.017 , a hard-vote video-level accuracy of 0.575 ± 0.100 , and a soft-vote video-level accuracy of 0.525 ± 0.050 across five seeds, consistently above the random baseline of 0.333 and the majority-class baseline of 0.375.

1.4 Related Work

Several works have explored computer vision in the context of foosball. Janssen et al. [2] propose a ball tracking method using 100 Hz computer vision on a semi-automated foosball table, showcasing that fast and robust tracking is achievable by efficient image processing. Horst et al. [3] present a real-time game state detector for an automated foosball table, using a YOLOX-based object detector to detect figures and a ResNet18-based regressor to estimate rod rotation angles, achieving 60 fps inference. In related work, Hagens [4] develops a game state detection system using convolutional neural networks, using ResNet18, ResNet50, MobileNetV3 and EfficientNetV2 as feature extractors, evaluating the approach on a real-world semi-automatic table setup. While these works focus on low-level game state estimation such as ball position and figure orientation, none address the higher-level problem of predicting the outcome of a game. This thesis builds on these foundations and extends the scope to end-to-end winner prediction from a raw video using a combination of classical computer vision and a TCN-based sequence classifier.

1.5 Thesis Structure

The thesis is structured as follows. Chapter 1 introduces the motivation, problem statement, contributions, related work, and structure of the thesis. Chapter 2 reviews the fundamental theoretical background underlying the system. Chapter 3 describes the complete implementation of the pipeline. Chapter 4 reports and analyses the experimental results. Chapter 5 summarises the findings, discusses limitations, and suggests directions for future research.

2 Background

2.1 Camera Model and Lens Distortion

2.1.1 The Pinhole Camera Model and The Intrinsic Matrix

The pinhole camera model [5] explains the projection of a 3D world point $P = (X, Y, Z)$ onto a 2D image plane, where X and Y are the horizontal and vertical coordinates of the point in the world coordinate system and Z represents the distance of the point from the optical centre of the camera. Using similar triangles, the projected pixel coordinates (u, v) are given by [5]:

$$u = f \cdot \frac{X}{Z}, \quad v = f \cdot \frac{Y}{Z} \quad (2.1)$$

where f is the focal length. The division by Z is known as perspective division and this is the reason why farther away objects appear smaller in the image. In a real camera, the top-left corner of the image is considered as the origin of pixel coordinates instead of the optical centre and this consideration introduces an offset called the principal point offset (c_x, c_y) . Furthermore, a square pixel cannot be assumed and therefore two components of focal length, f_x and f_y , are considered for the horizontal and vertical directions respectively. The updated projection is given by [6]:

$$u = f_x \cdot \frac{X}{Z} + c_x, \quad v = f_y \cdot \frac{Y}{Z} + c_y \quad (2.2)$$

These four parameters in Equation 2.2 are represented as a matrix called the camera matrix or intrinsic matrix K , as it depends entirely on the internal geometry of the camera [5, 6]:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (2.3)$$

Figure 2.1 illustrates the pinhole camera model.

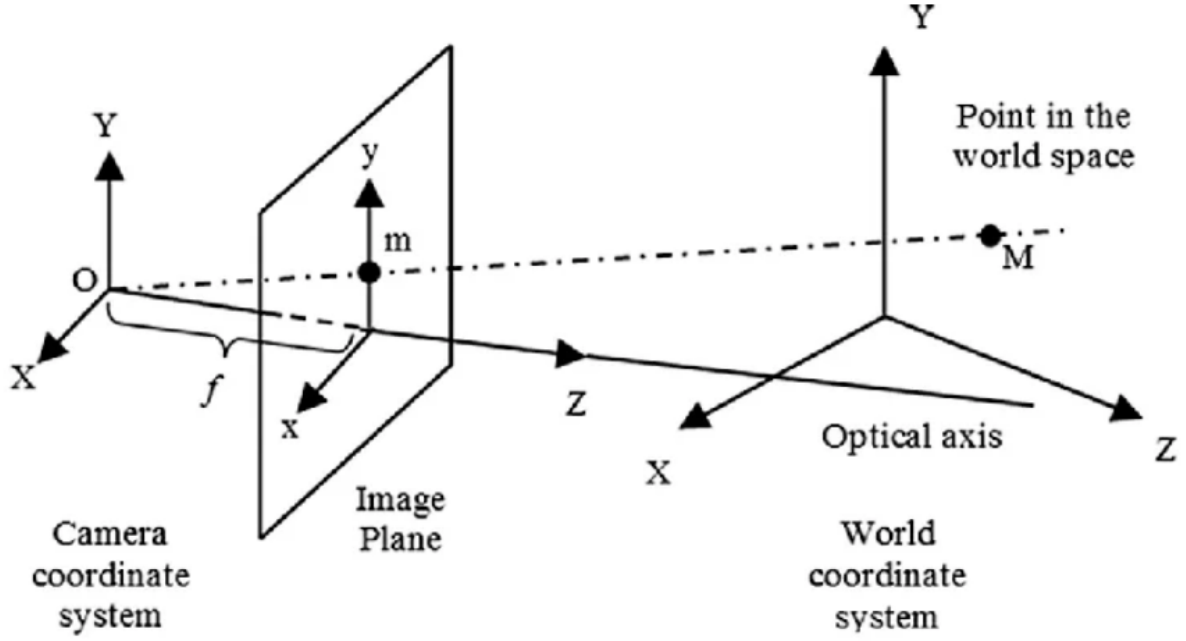


Figure 2.1: The pinhole camera model showing the projection of a 3D world point M (equivalent to P in this work) through the optical centre O onto the image plane at point m (equivalent to (u, v) in this work) taken from [7]

2.1.2 Lens Distortion

The assumption that light travels in a straight line through a single point fails in the real world, as the camera lenses are curved and this curvature bends the light, causing straight lines to appear curved in the image [5, 8]. The two dominant distortions are radial and tangential distortions.

1. **Radial distortion:** occurs when light passing through the edge of the lens bends more than light passing through the centre. Points are either pushed outwards, known as barrel distortion, or pulled inwards, known as pincushion distortion.
2. **Tangential distortion:** occurs when the lens plane is not perfectly aligned with the image sensor due to manufacturing imperfections. The image appears slightly tilted or skewed.

Figure 2.2 illustrates the effect of radial distortion on a regular grid pattern.

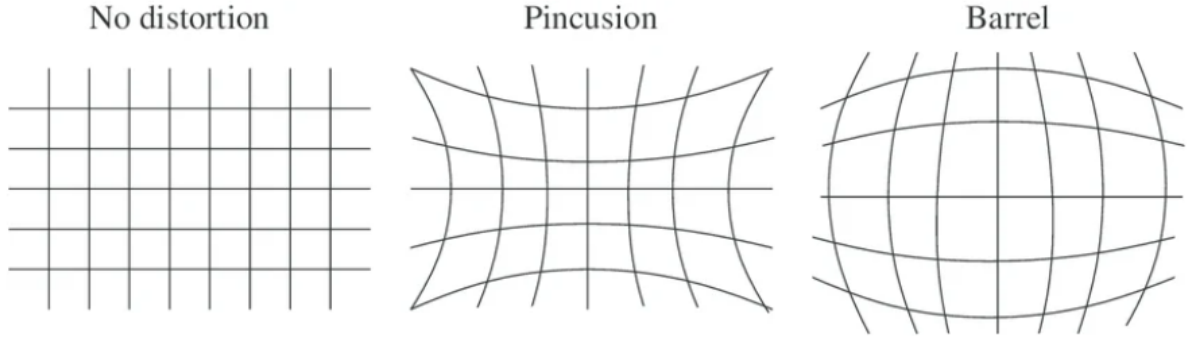


Figure 2.2: Illustration of barrel and pincushion distortion. Left: no distortion: straight grid lines remain straight. Centre: pincushion distortion: lines are pulled inward toward the centre. Right: barrel distortion: lines are pushed outward away from the centre. Image taken from [9].

The distortions are typically represented by their coefficients combined into a single vector known as the distortion vector [8]:

$$D = [k_1, k_2, p_1, p_2, k_3] \quad (2.4)$$

where k_1, k_2, k_3 are the radial distortion coefficients that describe how strong the distortion is at different distances from the centre, and p_1 and p_2 are the tangential distortion coefficients [8].

2.2 Camera Calibration

2.2.1 Zhang's Calibration Method

Zhang's calibration method [6] uses a physically printed checkerboard pattern as a calibration target because of how precisely the common meeting points between the black and white squares are detected. This board has known physical dimensions and each square is of the same size, which helps define the 3D coordinates of the inner corners relative to the board itself. Since the board is assumed to be flat, the world coordinates take the form $(X, Y, 0)$.

For every image captured, there exist two sets of points:

1. **3D object points:** these are the real-world inner corner coordinates and do not vary from image to image.
2. **2D image points:** these are the pixel coordinates of those same inner corners as detected in the image, and these positions vary from image to image.

A single image is not sufficient to recover K and D as the intrinsic parameters cannot be separated from the extrinsic parameters with a single viewpoint, making the problem geometrically degenerate [6]. A minimum of two images is required [6], however, the most effective way to ensure precise estimates of the coefficients is to collect as many images as possible and to make sure to have the board in different locations across different images such as edges, corners, close up, far away, tilted, and rotated, as every such orientation adds independent information for the solver.

Zhang’s method sets up a system of equations where for each corner in each image there exist two equations relating the known 3D corner coordinates to the detected 2D pixel coordinates through K , D , and the per-image extrinsics, as discussed in Section 2.1.1. These equations are nonlinear because the distortion coefficients in D appear as polynomial terms and the intrinsic and extrinsic parameters are coupled multiplicatively [6]. With multiple images and multiple corners per image, the number of equations rises to a high count and these equations are solved to find the four intrinsic parameters in K and five coefficients in D by minimising the total reprojection error across all images and corners simultaneously. The Levenberg–Marquardt algorithm [10] helps solve this system of nonlinear equations, which is discussed further in Section 2.3.2.

2.2.2 Corner Detection and Sub-pixel Refinement

Before using any of the OpenCV [11] functions on the extracted checkerboard images, it must be ensured that the images are sharp enough for accurate estimation of K and D . This is achieved by computing the Laplacian variance of the image [12]. The Laplacian operator computes the second derivative of the frame and responds strongly to the sharp edges of the checkerboard and weakly to smooth and blurry regions. The variance of the Laplacian across the whole frame is therefore a good measure of image sharpness, where a high variance corresponds to a sharp image.

To find the pixel coordinates of the inner corners, `cv2.findChessboardCorners` is used, which detects them automatically given the expected number of inner corners as input. It looks for saddle points in the image intensity i.e. points where the intensity goes from dark to bright in one direction and from bright to dark in the perpendicular direction simultaneously.

To further increase the accuracy of the estimates of K and D , the accuracy of the detected corner pixel coordinates must also be increased, because even a half pixel difference can adversely affect the estimates. This is achieved through sub-pixel refinement using `cv2.cornerSubPix` [6], which creates a window around each detected corner, computes the image gradients, and iteratively adjusts the corner position until the dot product of the gradient vector and the vector from the corner to its neighbours is minimised.

2.2.3 Reprojection Error

To evaluate the calibration quality, it is necessary to measure how accurate K and D are and this is achieved by reprojection [6]. Using the newly computed K and D , the known 3D object points are projected back onto the image and then the distance between these reprojected points and the points detected during the computations is measured. This distance is the reprojection error and in an ideal case of perfect calibration, these points coincide, i.e. reprojection error is zero. The RMS reprojection error is calculated across all corner points to obtain a single value which summarises the calibration quality.

2.2.4 Undistortion Maps

Obtaining an undistorted image using the distortion correction formula directly is computationally expensive as it requires processing every pixel individually. Therefore, lookup tables called undistortion maps are generated instead.

The optimal camera matrix K_{new} is first computed using `cv2.getOptimalNewCameraMatrix`, which takes K , D , and an `alpha` parameter as inputs. The `alpha` parameter controls how much of the undistorted image is retained. `alpha=1` keeps all pixels including the black border pixels that appear at the edges after undistortion, while `alpha=0` eliminates these black border pixels, retaining only valid pixels. In this work `alpha=0` is used.

Using K , D , and K_{new} , the undistortion maps `map1` and `map2` are precomputed by `cv2.initUndistortRectifyMap`. These maps store for every output pixel its corresponding source coordinates in the distorted image. The entire image can then be undistorted efficiently using `cv2.remap` by looking up the source coordinates rather than computing them per pixel.

2.3 Pose Estimation and Perspective-n-Point Problem

2.3.1 ArUco Fiducial Markers

Whenever a camera looks at a scene, it has no information about the geometry of the environment and therefore markers such as Fiducial markers are placed in the scene that help the computer understand its own position [13]. These markers are used as landmarks because they have known real-world coordinates. When these markers are detected in the image, the pose of the camera is recovered using the relationship between the known 3D world coordinates and the detected 2D image coordinates [13].

An ArUco marker has a thick black border and a binary pattern encoded inside. This binary pattern consists of black and white cells arranged in a grid and encodes a unique ID. The four corners of the outer black border are detectable with high precision [13].

ArUco markers are organised into dictionaries. For example, `DICT_4X4_50` means the binary pattern is arranged on a 4×4 grid and the dictionary contains up to 50 unique IDs [13].

Each marker contributes one anchor point and therefore four markers provide four anchor points in total. Each anchor point has a known 3D world coordinate and a corresponding detectable 2D image coordinate. These four matched pairs are the input required to solve the camera pose using the Perspective-n-point solver [14].



Figure 2.3: Examples of markers of different sizes, n , generated with the method proposed in [13]. From left to right: $n = 5$, $n = 6$ and $n = 8$.

2.3.2 The Perspective-n-Point Problem and Levenberg–Marquardt Refinement

The Perspective-n-Point problem, known as PnP, solves and recovers the position and orientation of the camera in space [14]. When given n points with known 3D world coordinates and their corresponding 2D image coordinates, the solver recovers the rotation and translation of the camera relative to the world coordinate system. The rotation is represented as a 3×1 vector `rvec` in the Rodrigues representation, where the direction of the vector is the axis of rotation and its magnitude is the angle of rotation in radians. The translation is represented as a 3×1 vector `tvec` giving the position of the world origin in the camera coordinate system. Together these two vectors fully define the position and orientation of the camera in space and are used to project 3D world points onto the image plane using `cv2.projectPoints`. Figure 2.4 illustrates the PnP problem.

OpenCV [11] provides several PnP solvers and one such solver is the `SOLVEPNP_ITERATIVE`, which uses an iterative refinement approach. It starts from an initial linearised estimate and then refines it by iteratively minimising the reprojection error [14]. With four point correspondences, the problem is minimally constrained and each point contributes two equations representing u and v which give eight equations in total, derived from Equation 2.2. These equations are just enough to solve for the six unknowns representing the rotation and translation vectors.

The solution from `cv2.solvePnP` is a good starting point but it is not fully optimal as the internal refinement uses a fixed number of iterations. To obtain a more accurate

solution, `cv2.solvePnPRefineLM` performs a dedicated optimisation pass directly on the reprojection error.

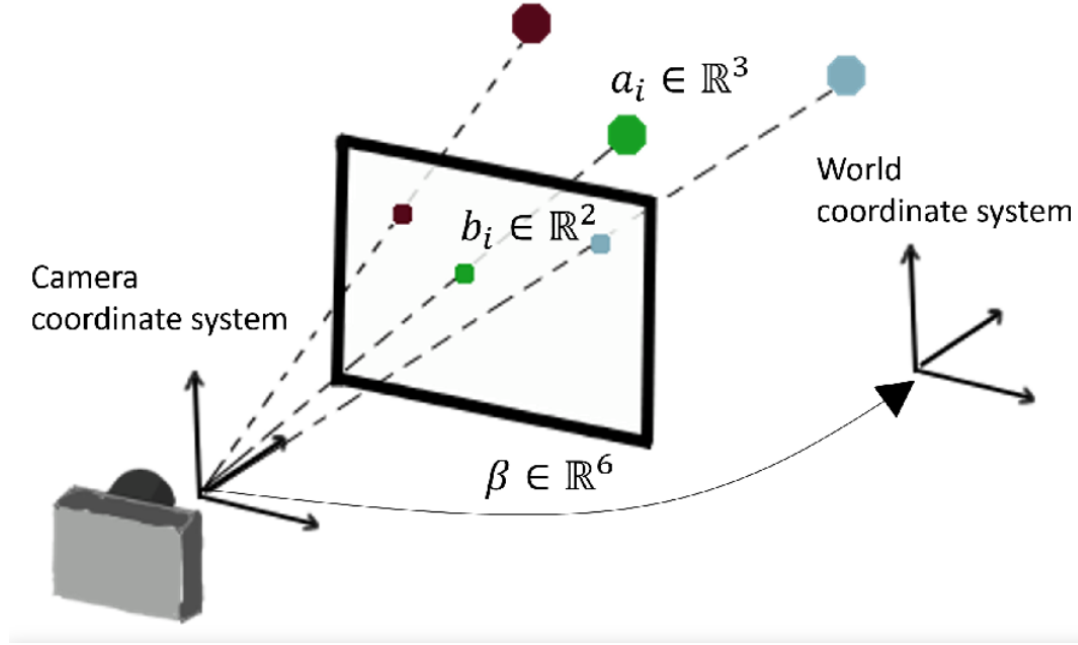


Figure 2.4: Illustration of the Perspective-n-Point problem. 3D world points $a_i \in \mathbb{R}^3$ are projected onto the image plane as 2D points $b_i \in \mathbb{R}^2$. The goal is to recover the camera pose $\beta \in \mathbb{R}^6$ consisting of three rotation and three translation parameters. Image taken from [15].

`cv2.solvePnPRefineLM` takes the initial `rvec` and `tvec` from `cv2.solvePnP` as the starting point and performs Levenberg–Marquardt optimisation [16]. This is a standard method for nonlinear least squares problems. It minimises the sum of squared reprojection errors, i.e. the squared distances between the projected 3D world points and the detected 2D points, by iteratively updating the solution. The algorithm chooses between two methods depending on how close it is to the minimum. When it is far from the minimum it uses gradient descent and when it is close to the minimum it uses the Gauss–Newton method [16]. The reason to use two different methods in a single optimisation lies in how these two methods work. Gradient descent is reliable and converges when it is far away but is extremely slow when it is near the minimum and on the other hand, Gauss–Newton method is fast near the minimum but the solution can diverge when it is far from minimum [16]. Combining both methods gives a solution that converges reliably and accurately, producing a refined `rvec` and `tvec` that precisely define the pose of the camera [16].

2.4 Homography and Perspective Correction

2.4.1 Projective Transformations and Homography

A homography is a 3×3 matrix H that represents a projective transformation between two planes [6]. Following the notation of [6], a point on the model plane $\mathbf{M} = [X, Y]^T$ and its corresponding image point $\mathbf{m} = [u, v]^T$ are related through this matrix. Expressing both points in homogeneous coordinates as $\tilde{\mathbf{M}} = [X, Y, 1]^T$ and $\tilde{\mathbf{m}} = [u, v, 1]^T$, the projection relationship is given by [6]:

$$s\tilde{\mathbf{m}} = H\tilde{\mathbf{M}} \quad (2.5)$$

where s is an arbitrary scale factor and $H = A[r_1 \ r_2 \ t]$ is the 3×3 homography matrix defined up to a scale factor [6]. Since H is defined up to scale, it has $9 - 1 = 8$ degrees of freedom [6]. Each point correspondence provides two equations and therefore a minimum of four point correspondences are needed to solve for the eight unknowns [6].

An important assumption in homography is the planar scene assumption. A homography is only valid when all mapped points lie on a single plane in 3D space [6]. If points are not coplanar, a single homography matrix cannot represent the mapping and therefore different regions of the scene at different depths would each require a different homography.

2.4.2 Homography Estimation and Perspective Warping

When the four matched point pairs are known, the homography matrix H is estimated by stacking the correspondence equations into a linear system [6]:

$$L\mathbf{x} = \mathbf{0} \quad (2.6)$$

where L is a $2n \times 9$ matrix constructed from the point correspondences and $\mathbf{x} = [h_1^T, h_2^T, h_3^T]^T$ is the vectorised form of H [6]. The solution of this equation is the eigenvector of $L^T L$ which corresponds to the smallest eigenvalue and this is achieved using Singular Value Decomposition [6]. The Direct Linear Transformation finds the H that minimises the algebraic error across all point correspondences [6].

Once H is known, `cv2.warpPerspective` applies it to the entire image to produce a perspective corrected output. For every pixel (x', y') in the output canvas, the inverse homography H^{-1} is applied to find the corresponding source position (x, y) in the original image. The pixel color at that source position is sampled and placed at (x', y') in the output. The inverse mapping is used rather than forward mapping to avoid holes in the output canvas. The result is a warped bird's-eye view of the scene in which perspective

distortion is completely eliminated. Figure 2.5 illustrates perspective correction.

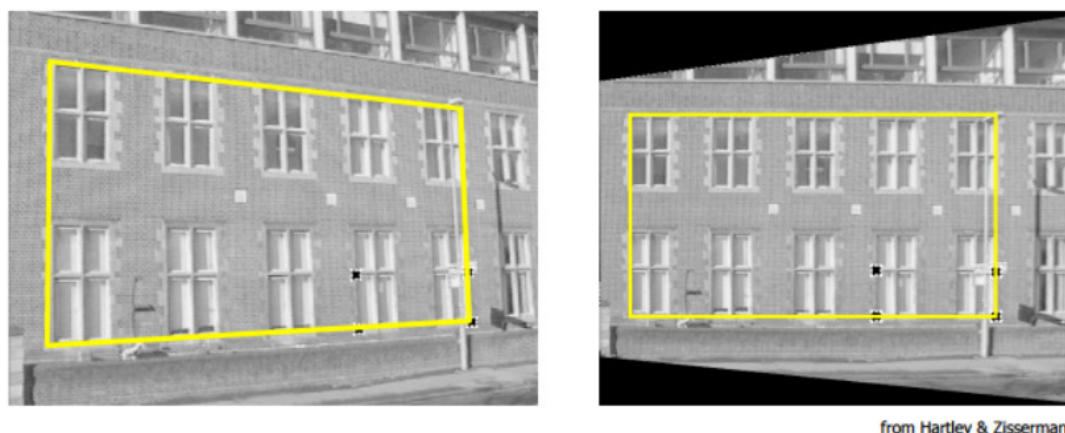


Figure 2.5: Example of perspective correction using homography. Left: original image with a quadrilateral region defined by four point correspondences. Right: the warped output after applying the homography, producing a frontal view free of perspective distortion. Image taken from [17].

2.5 Color Spaces and HSV-Based Detection

2.5.1 BGR vs HSV Color Representation

BGR, which stands for Blue, Green and Red, is OpenCV's [11] default color format and each of these values ranges from 0 to 255. One of the key limitations of this format is that all three values change when there is a change in lighting. For example, a red ball under two different lighting conditions has two different sets of B, G, and R values, making it difficult to define a stable range for a detection task.

This is where HSV comes into the picture [18]. HSV has three components:

- **Hue:** carries information about the actual color and ranges from 0 to 179 in OpenCV [11].
- **Saturation:** represents how vivid or washed out the color is and ranges from 0 to 255.
- **Value:** represents how bright or dark the color is and ranges from 0 to 255.

When there is a moderate change in lighting, H remains approximately stable while S and V change significantly. Since H carries the color information and remains within a stable range, it can be reliably used for color-based object detection [18]. Figure 2.6 illustrates the HSV color space as a cone. Note that in OpenCV [11], the Hue range is scaled to 0–179 instead of the standard 0–360°, but the geometric representation remains the same.

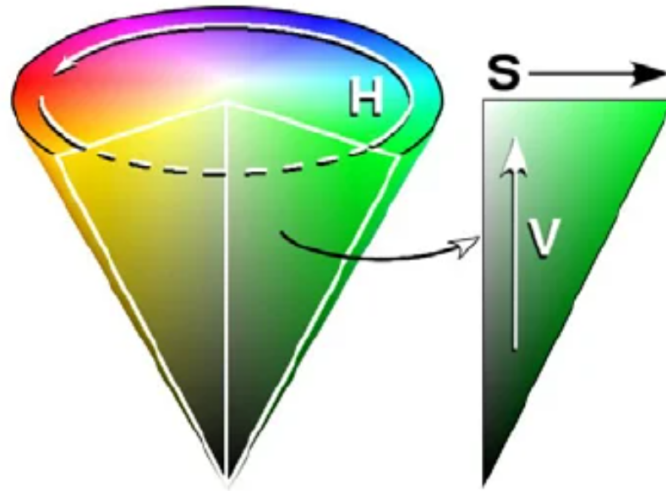


Figure 2.6: The HSV color space represented as a cone. Hue H is encoded as the angle around the cone, Saturation S as the distance from the central axis, and Value V as the height along the vertical axis. All colors collapse to black at the tip of the cone where $V = 0$. Image taken from [19].

2.5.2 HSV Thresholding for Ball Detection

The OpenCV [11] function `cv2.inRange` takes the lower and upper HSV bounds as input and generates a binary mask. This mask has only two values, white (255) and black (0). All pixels whose HSV values fall within the defined range are represented as white and the rest as black.

An interesting challenge specific to the color red is the hue wrap-around problem. In OpenCV [11], red appears on both ends of the hue spectrum i.e. near 0 and near 179. This is handled by a wrap-around check. If the minimum and maximum sampled hue values are found on both ends of the spectrum simultaneously, the range is switched to a wide range spanning both ends to ensure no red pixels are missed when traversing from lower to upper.

Tolerance padding is also added to the lower bounds of S and V to account for lighting variation.

2.5.3 Morphological Cleaning

After thresholding, a binary mask is generated containing white blobs from all red objects in the scene. To further refine the detection, two morphological operations are applied to this mask [20]:

1. **Erosion** : shrinks all white regions. Small white blobs generated as a result of noise

are reduced to zero or eliminated entirely. Larger blobs from actual objects in the scene shrink too but survive the operation [20].

2. **Dilation** : expands all white blobs. It can only expand blobs that survived the erosion step. In this way, smaller fragments of the same blob are merged back together to form one larger connected blob [20].

This sequence where erosion is applied first followed by dilation is called morphological opening. The reverse sequence, dilation followed by erosion, is called morphological closing [20].

Figure 2.7 illustrates both operations on a binary image.

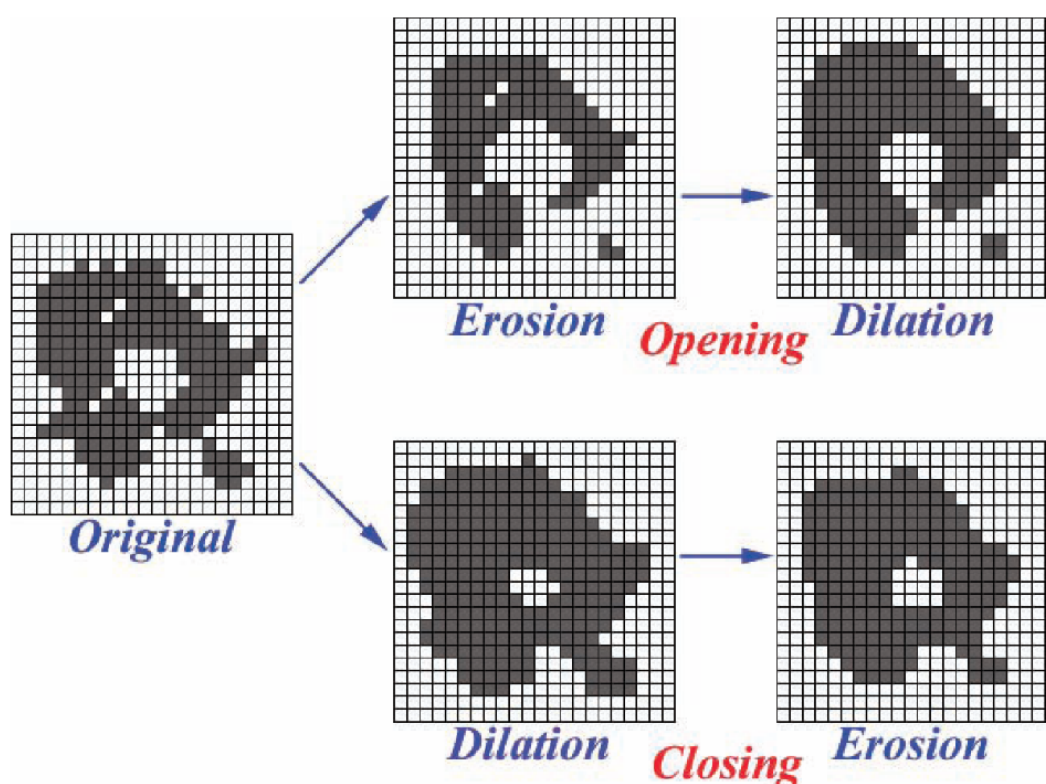


Figure 2.7: Illustration of morphological opening and closing operations on a binary image. The top path shows erosion followed by dilation, which produces the opening result. The bottom path shows dilation followed by erosion, which produces the closing result. Image taken from [21].

2.6 Contour Detection and Blob Fitting

2.6.1 Contour Extraction

After the morphological operations, a clean binary mask is available for contour extraction. The OpenCV [11] function `cv2.findContours` finds the outlines of all blobs in the clean

mask. Two flags are used:

- `RETR_EXTERNAL` : returns only the outermost contours, ignoring any nested contours inside larger blobs.
- `CHAIN_APPROX_SIMPLE` : stores only the endpoints of horizontal, vertical, and diagonal segments rather than all intermediate points, which reduces memory usage.

2.6.2 Minimum Enclosing Circle

The OpenCV [11] function `cv2.minEnclosingCircle` fits the smallest circle that contains all contour points and returns the centre (x, y) and the radius of the circle. This centre acts as the position estimate of the detected object. The radius is used as a size filter to reject false detections from other objects in the scene such as shoes, jerseys, or any other red objects in the background. Only contours with a radius within a defined range are accepted as valid ball detections.

2.7 Kalman Filter for Object Tracking

The Kalman filter is a mathematical tool that estimates the state of a moving object by combining two sources of information, i.e. predictions obtained from a motion model and noisy measurements obtained from the camera [22, 23].

2.7.1 State-Space Representation

For a 2D tracking problem, consider an example of a state vector comprising four variables carrying information about position and velocity [24]:

$$\mathbf{x} = [x, y, v_x, v_y]^T \quad (2.7)$$

where x and y are the position coordinates, and v_x and v_y are the velocities in the horizontal and vertical directions respectively.

The transition matrix describes how the state evolves from one frame to the next, assuming the velocity remains constant between frames. This is known as the constant velocity model [22–24]:

$$F = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.8)$$

Multiplying this matrix by the state vector generates four equations:

$$x_{new} = x + v_x, \quad y_{new} = y + v_y, \quad v_{x,new} = v_x, \quad v_{y,new} = v_y \quad (2.9)$$

The measurement matrix defines what can actually be observed from the camera [22, 23]:

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \quad (2.10)$$

Only position x and y can be observed directly and the velocities, v_x and v_y , are estimated internally by the filter [23, 24].

2.7.2 Prediction and Correction Steps

Each frame involves two steps: predict and correct [22, 23].

Predict step: Before any detection occurs in the current frame, the filter predicts the ball position based on the previous state and the transition matrix [22]:

$$\hat{\mathbf{x}}_{k|k-1} = F \cdot \mathbf{x}_{k-1} \quad (2.11)$$

Simultaneously, the prediction uncertainty P is updated [22]:

$$P_{k|k-1} = F \cdot P_{k-1} \cdot F^T + Q \quad (2.12)$$

This ensures that a position estimate is always available even before any detection occurs in the current frame.

Correct step: If the ball is detected in the current frame, the measured coordinates $z_k = [x_{detected}, y_{detected}]^T$ are available. The filter updates its state by combining the prediction with the measurement [22]:

$$\mathbf{x}_k = \hat{\mathbf{x}}_{k|k-1} + K_g \cdot (z_k - M\hat{\mathbf{x}}_{k|k-1}) \quad (2.13)$$

The term $(z_k - M\hat{\mathbf{x}}_{k|k-1})$ is the innovation i.e. the difference between what is predicted and what is actually measured [23]. In the ideal case of a perfect prediction, the innovation is zero and the state does not change.

K_g is the Kalman gain and controls how much weight is given to the prediction versus the measurement [23]. In this 2D tracking case, it acts as a scalar-like weighting factor where a value closer to 0 means the filter trusts the prediction more and a value closer to 1 means it trusts the measurement more.

The Kalman gain is computed as [22]:

$$K_g = \frac{P \cdot M^T}{M \cdot P \cdot M^T + R} \quad (2.14)$$

where P is the prediction uncertainty, M is the measurement matrix, and R is the measurement noise covariance reflecting confidence in the camera detection. Q is the process noise covariance reflecting confidence in the motion model. A large Q causes P to grow faster, as seen in Equation 2.12, increasing K_g and making the filter trust the measurement more. A small Q keeps P small, decreasing K_g and making the filter trust the prediction more. R has the opposite effect i.e. a large R decreases K_g and a small R increases K_g , as seen in Equation 2.14. In summary, Q and R together determine the Kalman gain through P , and the specific values used are discussed in Section 3.6.

2.7.3 Tracking Under Occlusion

When the ball is not detected, the correction step is completely skipped as there is no measurement to compare against and only the prediction step is triggered [24]. The filter keeps predicting using the transition matrix and the last known velocities, essentially extrapolating the ball position forward.

Without the Kalman filter, every occlusion would be treated as a case of complete tracking loss, creating trajectory gaps in the data generated downstream.

2.8 1D Signal Processing for Rod Detection

2.8.1 The Bilateral Filter

Whenever signals are processed, smoothing is necessary and many filtering options exist. The main intention of filtering is to smooth the signal and two filters, namely Gaussian and bilateral, are discussed. In a Gaussian filter, the value of the centre pixel is computed as a weighted average of its neighbours. The weights are determined based on how close or how far the pixels are. If it is close, a high weight is assigned and if it is far, a low weight is assigned. This is called weight determination by spatial distance. The weights follow a Gaussian bell curve. The problem with such a filter is that it does not care about the content of the image. It smooths everything uniformly and blurs the sharp edges and therefore edge location becomes difficult. The bilateral filter addresses this limitation. This filter has two weighting criteria which are applied simultaneously. The first weight is the same as the Gaussian filter, determined by spatial distance. The second weight is determined by intensity similarity [25]. Neighbours whose intensity values are similar to the centre pixel are assigned high weights and neighbours whose intensity values are different are assigned low weights. The combined weight for the bilateral filter is therefore [25]:

$$w(i, j, k, l) = \exp\left(-\frac{(i-k)^2 + (j-l)^2}{2\sigma_{space}^2}\right) \cdot \exp\left(-\frac{|I(i, j) - I(k, l)|^2}{2\sigma_{color}^2}\right) \quad (2.15)$$

where (i, j) is the centre pixel, (k, l) is a neighbour, and I is the intensity value. There are three parameters in the bilateral filter: d , σ_{space} and σ_{color} [25]. The parameter d is the window size and defines how many pixels around the centre are considered when the weights are computed. σ_{space} controls how large the spatial neighbourhood is. A higher value means far neighbours have significant weight and a lower value means only close neighbours have significant weight [25]. σ_{color} controls the intensity similarity criterion [25]. A higher value means neighbours with quite different intensities are included in the averaging process, making the filter behave more like a Gaussian filter. A lower value means only neighbours with very similar intensity are included, making the filter strictly edge-preserving [25].

2.9 Temporal Convolutional Networks

A Temporal Convolutional Network (TCN) is a convolutional architecture designed for sequence modelling [1]. The architecture proposed by Bai et al. combines dilated causal convolutions, residual connections, weight normalisation, and dropout to efficiently capture long-range temporal dependencies. In this work, weight normalisation is replaced by batch normalisation.

2.9.1 Sequence Classification

A sequence is an ordered collection of observations where the order carries meaning [26]. In a regular dataset each sample is independent, but in a sequence the position of each observation relative to the others matters. When the order of observations changes, the meaning also changes entirely. Examples of sequences include audio signals, video frames, sensor values, stock prices, and so on [26]. Classification of a sequence is a supervised learning task where the goal is to assign a single label to each sequence. The input is a fixed-length series of observations and the output is a single label from a fixed set of classes. The challenge is to summarise the temporal evolution of the sequence into a single class label. Naive approaches that ignore this order completely destroy the temporal information and therefore cannot capture meaningful patterns in the data. [26].

In some cases sequences may have different lengths depending on the observations and therefore a fixed length across all samples is needed because a neural network expects a fixed size input. This is achieved by zero padding i.e. shorter sequences are padded with zero values. The global average pooling layer at the end of the TCN reduces the effect of

padded zeros by averaging across the full sequence length, though the padded positions still contribute to the mean.

2.9.2 Dilated Causal Convolutions

A one-dimensional convolution slides a filter of size k across the sequence and at each position computes a weighted sum of the values under the filter [27]. Unlike 2D convolutions used for images, 1D convolutions operate along the time or sequence dimension only, making them suitable for temporal data. The output at each position is influenced by k consecutive input values. The number of input values the filter considers is the receptive field. For example, when $k = 3$, three values are considered at a time. The problem with this is that longer sequences require a very large receptive field, but stacking more layers to increase the receptive field slows the model down and is expensive to compute.

A simple solution for this is dilation [28]. Instead of reading consecutive values, some values in between are skipped. With a dilation of one, all values are read consecutively without skipping anything. With a dilation of two, every second value is read, i.e. positions one, three, five, seven and so on. With a dilation of four, every fourth value is read, i.e. positions one, five, nine and so on. Doubling the dilation value directly doubles the receptive field without adding any parameters. By stacking layers with dilations one, two, four and so on, the receptive field grows exponentially and the problem of requiring a large receptive field is solved. The causal concept added along with dilation ensures that the model only looks at past and present values [29]. This is achieved by padding only the left side of the sequence [1]. Figure 2.8 illustrates dilated causal convolutions.

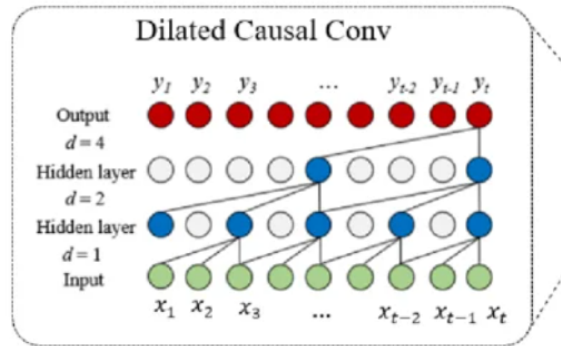


Figure 2.8: Dilated causal convolutions with dilation factors $d = 1$, $d = 2$, and $d = 4$. Each output y_t depends only on current and past inputs, enforcing causal behaviour. Image taken from [30].

2.9.3 Residual Connections

When the number of layers stacked in a network increases, it becomes more and more difficult to train the model. The gradient signal, which tells the model how to update its weights, diminishes as it travels backward through the layers. By the time it reaches the early layers it is almost zero and the early layers stop learning. This is called the vanishing gradient problem [31]. The solution adopted to address this problem is called a residual connection [32]. Instead of sending the output of a layer directly to the next layer, the input x is added directly to the output of the learned transformation:

$$\text{output} = F(x) + x \quad (2.16)$$

This creates a shortcut path through which the gradient signal flows directly without passing through the learned weights. Early layers now receive a strong gradient signal irrespective of the depth of the network. When the number of channels changes between the input and output, a 1×1 convolution is used to match the dimensions before the addition [32]. Figure 2.9 illustrates a simple residual connection.

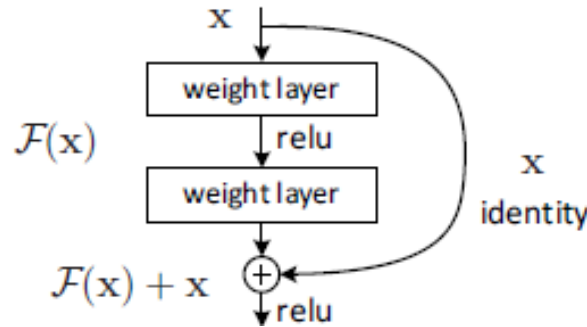


Figure 2.9: Residual connection in a TCN block. The input x is added to the output of the main path $F(x)$ before a final ReLU activation. Image taken from [32].

2.9.4 Batch Normalisation and Dropout

During the training process, as the weights change, the distribution of values flowing through the network keeps shifting. This makes training unstable and slow. This is solved by batch normalisation, which normalises the values at each layer to have a mean of zero and unit variance across the current batch. Following [33], the normalisation is defined as:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (2.17)$$

A learned scale γ and shift β are then applied to allow the network to undo the normalisation whenever needed. Following [33], this is defined as:

$$y = \gamma \hat{x} + \beta \quad (2.18)$$

This ensures the values remain stable throughout training and also allows learning rates to be higher and therefore this makes training faster and more reliable [33]. In this work, **BatchNorm1d** is used as the input is a 1D sequence rather than a 2D image.

Another technique called dropout randomly turns off a fraction of neurons during training [34]. For example, if the dropout rate is 0.4, then 40% of the neurons are turned off during each forward pass. During testing all neurons are active [34]. By adopting dropout, the network cannot rely on any single neuron always being available and is therefore forced to learn multiple independent ways to represent the same information. This prevents the model from overfitting and helps it generalise rather than memorise [34]. This is particularly important when dealing with small datasets. Figure 2.10 illustrates the dropout mechanism during training and inference.

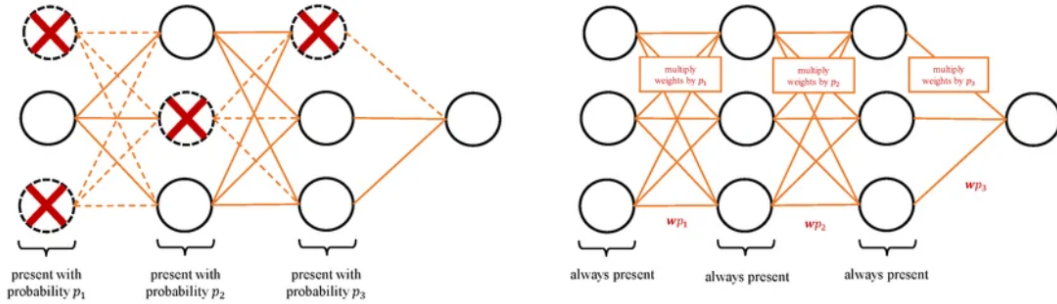


Figure 2.10: Dropout during training and inference. Left: during training, neurons are randomly dropped with probability p , forcing the network to learn. Right: during inference, all neurons are present. Image taken from [35].

2.9.5 Global Average Pooling

After the convolutional layers, the output has the shape (channels, sequence_length). To classify the sequence, a fixed size vector must be provided as input to the classifier. Global average pooling achieves this by taking the average of all values along the time dimension for each channel [36]. For example, if there are 16 channels and 300 time steps, the output is a vector of 16 values i.e. one average per channel. Figure 2.11 illustrates the difference between a fully connected layer after flattening and global average pooling.

This is better than flattening for two reasons:

- Flattening produces a vector of $16 \times 300 = 4800$ values and requires a much larger classifier layer
- Global average pooling fixes the representation size and keeps the number of parameters low [36].

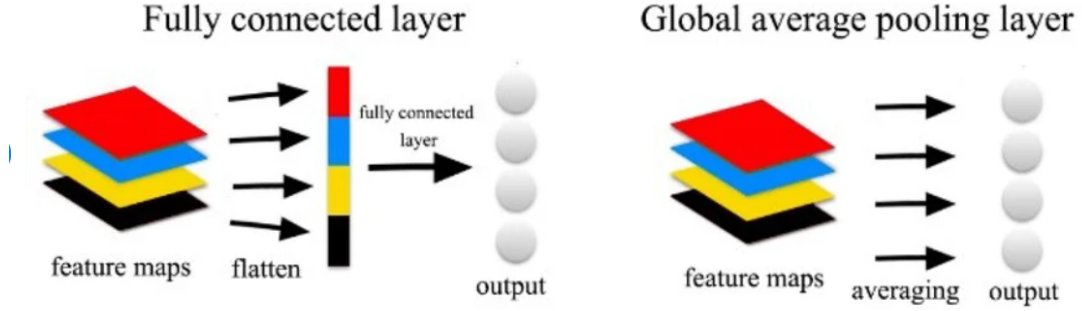


Figure 2.11: Comparison between a fully connected layer after flattening (left) and global average pooling (right). Image taken from [37].

2.9.6 Class Imbalance and Weighted Loss

A dataset is believed to have class imbalance when certain classes have more occurrences than others. For example, if there are three classes A, B, and C and 80% of the data belongs to class A, 15% to B, and 5% to C, the model becomes lazy and learns to always predict class A without actually learning from the sequence. Standard cross-entropy loss does not prevent such lazy predictions [38, 39].

Weighted cross-entropy is therefore used, which assigns higher penalties to mistakes on classes with fewer samples [38]. The weight of each class is computed as:

$$w_c = \frac{N}{C \cdot n_c} \quad (2.19)$$

where N is the total number of samples, C is the number of classes, and n_c is the number of samples in a class c . A class with fewer samples is assigned a higher weight [39]. The weighted loss function becomes:

$$L = - \sum_{c=1}^C w_c \cdot y_c \cdot \log(\hat{y}_c) \quad (2.20)$$

By incorporating this, the model is forced to pay more attention to the minority classes during training [38, 39].

2.10 Voting for Video-Level Prediction

2.10.1 Hard Voting and Soft Voting

Instead of classifying the whole video at once, classification is done at the sequence level independently. This means there are multiple predictions for the same video depending on the number of sequences. To obtain a single predicted class for the whole video, two types of voting are available: hard voting and soft voting. Figure 2.12 illustrates the difference between hard and soft voting.

In hard voting, each sequence casts a single class vote and the class with the most votes wins. This is simple but discards the confidence information available across the classes[39].

In soft voting, instead of a single class vote, the full probability vector of each sequence is available. The probability vectors are averaged and the class with the highest average probability is taken as the final prediction. This uses the full confidence information and is generally more reliable than hard voting[39].

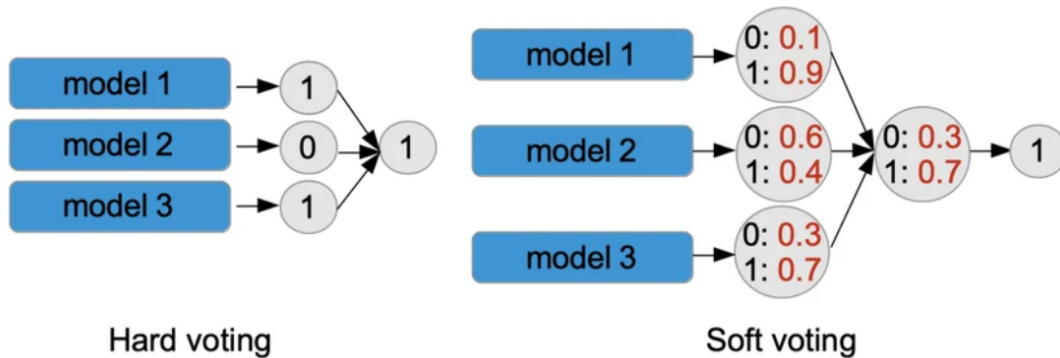


Figure 2.12: Hard voting (left) and soft voting (right). In hard voting, the class with the most votes wins. In soft voting, the class with the highest average probability is taken as the final prediction. Image taken from [40].

2.11 Multi-Seed Evaluation

Training of any neural network begins by choosing some random weights. Different starting points yield different models and different results, especially on small datasets. Therefore, a multi-seed approach is used where the model is trained using multiple different seeds to identify whether the results are consistent or sensitive to the choice of initialisation.

Training with different seeds and reporting the mean and standard deviation gives a clearer and more honest picture of how well the architecture performs. The mean is

the expected performance and the standard deviation tells how much the result varies depending on the random starting point. A high standard deviation indicates that the model is sensitive to initialisation and the results may not be reliable across different runs.

3 Method

3.1 System Overview

3.1.1 Hardware Setup

The foosball table used throughout the thesis belongs to the brand *SCORER* from *KICKERSPORT.DE*. Its playing field has a width of 1200 mm and a height of 680 mm, with one goal on each side. The table has two teams, namely Black and White, based on the color of the figurines. These figurines are mounted on eight rods and each of them has a fixed X position and a dynamic Y position. All figurines on a given rod move together as the rod is pushed. Each team has four rods, arranged as shown in Figure 4.11 in experiments section 4.5.3. The Black team occupies rods one, two, four and six, and the White team occupies rods three, five, seven, and eight counted from left to right. The rod roles are as follows: rods one and eight are goalkeepers with one figurine each, rods two and seven are defenders with two figurines each, rods four and five are midfielders with five figurines each, and rods three and six are attackers with three figurines each. This gives a total of 11 players per team and 22 players on the field. Figure 3.1 shows the ball used in this thesis, which is red in color and has a diameter of 35 mm.

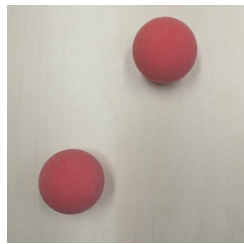


Figure 3.1: The red ball used throughout the recordings with a diameter of 35 mm.

The setup is upgraded for this project by constructing an aluminium frame on top of the field, on which a single USB webcam is mounted at a fixed overhead position pointing down at the field. The webcam used belongs to the brand *PLUSONIC* with a lens of 3.6 mm and is shown in Figure 3.2. The frame also houses 2 LED light sources, as shown in Figure 3.3, which provide consistent illumination of the field during video recording.

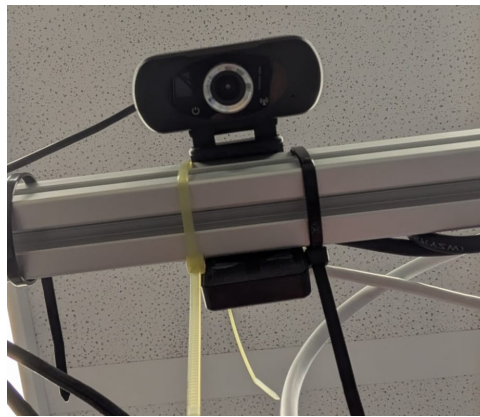


Figure 3.2: The PLUSONIC USB webcam with a 3.6 mm lens mounted on the aluminium frame at a fixed overhead position.

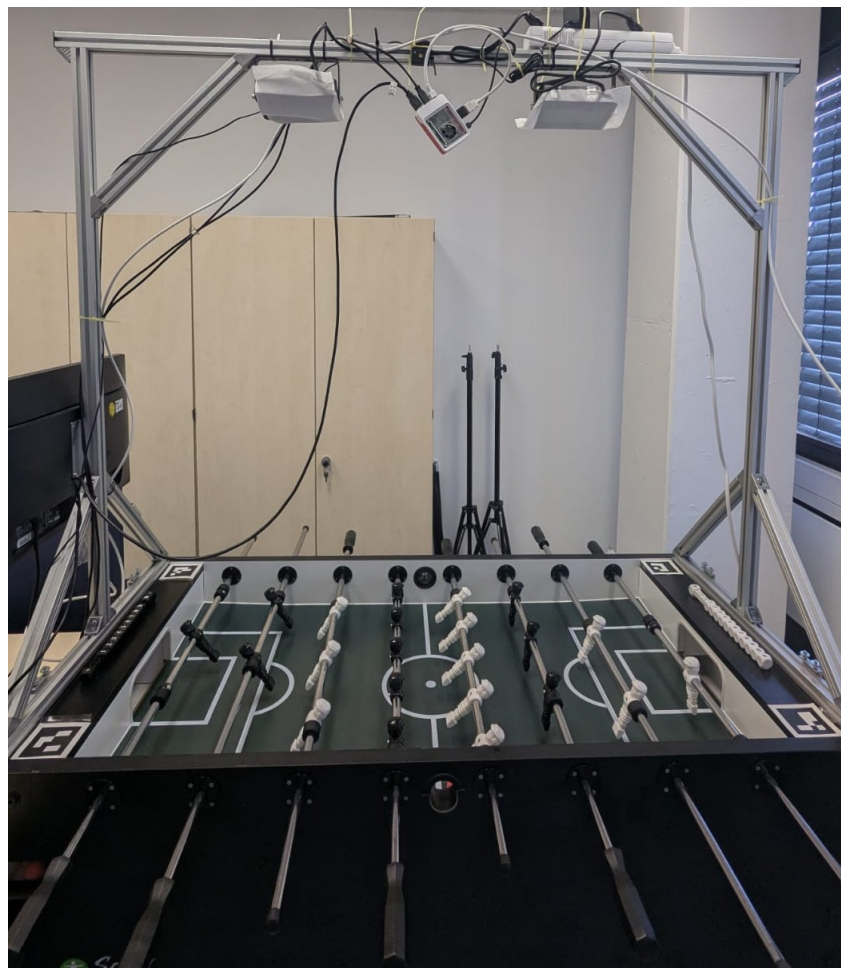


Figure 3.3: The complete experimental setup. An aluminium frame is mounted over the table football housing a single overhead USB webcam and two LED light sources that provide consistent illumination of the playing field.

3.1.2 Pipeline Architecture

The architecture of the pipeline is in the shape of a reverse Y or it can be considered as a shared branch architecture. Every video undergoes the same process in the shared trunk before it forks into 2 independent branches.

Shared trunk: undistortion, perspective correction, ball detection and tracking, and player position detection.

Branch 1 — Hit Detection: hit zone evaluation, entry/exit state machine, and hit classification. The output CSV contains the following columns: `Frame`, `Timestamp`, `X`, `Y`, `Speed_m_s`, `Accel_m_s2`, `Status`, `Goals_Black`, `Goals_White`, `Hit_Zone`, `Hit_Successful`, `Hit_Player`, `Hit_Reason`, `Hit_Category`, `Hit_Team`, `y_rod_1_p1` through `y_rod_8_p1`, `x_ball(normalised)`, `y_ball(normalised)`, `ball_detected`, `frame_norm`.

Branch 2 — Learning Pipeline: feature extraction and normalization. The output CSV contains: `x_ball`, `y_ball`, `y_rod_1_p1` through `y_rod_8_p1`, `Status`, `frame_norm`. The `Status` column is eliminated during post-processing after it is used as a filter to eliminate all rows except for on field and occluded.

3.1.3 Image Representations Used Throughout the Pipeline

The pipeline operates across three progressively refined image representations. Each representation addresses one limitation of the previous one and together they form the basis for all subsequent processing steps.

System 1: Raw camera pixels: These are the raw pixels generated by the camera and they are distorted with various types of distortions discussed in Chapter background 2.1.2. The straight lines appear to be curved and the objects are scaled based on the distance from the camera. Therefore this coordinate system is not reliable.

System 2: Undistorted image pixels: After the camera is calibrated and the entire image is undistorted, the straight lines now appear to be straight but one key unwanted property called perspective still exists. This makes the far end appear smaller and the distances measured in this coordinate system are not physically reliable.

System 3: Warped bird's-eye view: After doing perspective correction i.e. performing the homography, a top-down view is obtained on a canvas of 1300×780 pixels with a 50 mm padding included on all sides. Since the homography is designed so that the four corners of the 1200×680 mm field map exactly to the corners of the 1200×680 pixel interior region, each pixel corresponds exactly to 1 mm.

The reason for working with the third system i.e. the warped view is that the perspective distortion is eliminated and the physical distances now have meaning.

3.2 Camera Calibration

3.2.1 Calibration Video Recording

The aim of this step is to record a high quality raw video for camera calibration. It is important to maintain a uniform resolution across the entire project as K and D are resolution dependent and if the resolution differs between calibration and game recording, the focal lengths f_x and f_y would be incorrect for the actual use case. The checkerboard used is of size 10×7 squares and therefore has 9×6 inner corners, with each square measuring $71\text{mm} \times 71\text{mm}$.

The recording begins with a warm-up phase in which the first 30 frames are discarded to allow the camera to stabilise before recording begins. To be able to extract up to 40 high quality calibration images after filtering, the total number of recorded frames is set to 5000. While recording is in progress, it is crucial to hold the checkerboard in different positions and orientations i.e. moving it to all four corners of the frame, tilting it up and down, tilting it left and right, rotating it, and varying its distance from the camera. Although a manual snapshot approach would also work, a continuous recording is preferred as it is faster, consistent, and not prone to human error, and the frames can be automatically extracted using quality filters.

3.2.2 Calibration Image Extraction and Quality Filtering

Though 5000 frames are available for calibration, most of them are redundant or blurry and do not qualify for calibration. Therefore, filters are applied to extract high quality images, with a minimum of 10 and a maximum of 40 images saved.

Filter 1: Sharpness using Laplacian Variance: Since there is no point in detecting corners on blurry images, this filter classifies each image as sharp or blurry and takes a decision to either retain or discard it. It computes the Laplacian variance for each image as discussed in Section 2.2. The threshold is set to 100 and images with a variance below 100 are classified as blurry and discarded.

Filter 2: Motion Stability: Even though an image is classified as sharp by the first filter, the frame might have been captured while the board was in motion. This filter removes such frames with smeared edges. This is achieved by maintaining a sliding window of four consecutive frames and computing the mean displacement of all 54 detected corners between these consecutive frames. If the mean displacement across all three consecutive frame pairs is below 1.0 pixel, the board is classified as stationary and the frame is retained.

Filter 3: Spatial Diversity: Despite a frame being classified as sharp and stationary,

it might not carry any new information for calibration if it is in a similar position to the previously saved frame. This filter ensures that two consecutively saved frames are spatially apart. The distance between the board centres of two consecutively saved frames is computed and only frames whose distance exceeds 30 pixels are accepted.

In addition to these three filters, the board size in pixels is also tracked across all saved images. A large board size in pixels indicates that the board is close to the camera, while a small board size indicates it is far away. Tracking this ensures that the frames used for calibration cover a range of distances, as distortion at different depths needs to be sampled for an accurate calibration.

3.2.3 Calibration Solve and Iterative Refinement

Once the images are filtered, the core calibration functions are called to detect the corners. To ensure robust detection under uneven lighting, the flags `CALIB_CB_ADAPTIVE_THRESH` and `CALIB_CB_NORMALIZE_IMAGE` are used in `cv2.findChessboardCorners` to improve detection. This is followed by the refinement function `cv2.cornerSubPix` with a window size of 11×11 , iterating over a maximum of 50 times with a tolerance of 10^{-3} , to obtain the corners with subpixel accuracy. `cv2.calibrateCamera` then takes the 3D object points scaled to millimetres and the corresponding 2D image points and using Zhang's method solves for K and D . The per-image reprojection errors obtained from the calibration are then sorted from highest to lowest and the two images corresponding to the worst values are discarded. The calibration process is then carried out once again to obtain refined and accurate K and D . Using these K and D , the optimal camera matrix K_{new} is computed with `cv2.getOptimalNewCameraMatrix` taking `alpha=0.0` and the undistortion maps `map1` and `map2` are precomputed and saved. These calibration artefacts are loaded at the start of whole detection and tracking pipeline to undistort every raw frame before any processing begins.

3.2.4 Calibration Validation

To avoid blind acceptance of the obtained K and D , a series of sanity checks are performed to verify that the recovered parameters are physically plausible.

Check 1 — Principal Point: c_x and c_y are extracted from the intrinsic matrix K and checked against the expected image centre. For a 640×480 image the expected centre is (320, 240). If the deviation is within 30 pixels in either direction, the principal point is considered accurate and the calibration is considered good.

Check 2 — RMS Reprojection Error: The final RMS reprojection error is checked against two thresholds. It is classified as excellent if below 0.5 pixels and acceptable if below 1.0 pixel. A warning is displayed if it exceeds 1.0 pixel.

Check 3 — Visual Comparison: After the three sanity checks, a visual verification is performed to confirm that the undistortion is working correctly. A calibration frame is compared side by side with its undistorted counterpart to visually confirm that the distortion has been correctly removed.

3.3 Data Recording

3.3.1 Dataset Construction and Labeling

Each recorded video is 60 seconds long at 30 fps, giving 1800 frames per video. A total of 57 videos are recorded with different players playing the game. A total of seven players play the game in different combinations. These 57 videos are used as-is without further processing for Branch 1, i.e. the Hit Detection branch.

For the Learning Pipeline branch, each video is labelled during post-processing based on the number of goals scored, by looking at the goal counts produced by the tracker. These labels are stored in a dictionary during post-processing before the training process.

3.3.2 Train/Test Split Design

The train and test split is designed such that no video appears in both sets. If sequences from the same video were split across train and test, this could lead to several problems:

1. The same players and playing style appear in both sets
2. The same lighting conditions are shared
3. Sequences share game context, i.e. sequence four is influenced by sequences 1–3
4. The model may learn to recognise specific player movements, causing data leakage

Players may still repeat across train and test, but the game dynamics are different.

3.4 Perspective Correction

3.4.1 3D World Coordinate System Definition

Before any measurements or projections are carried out, the coordinate system is defined. A 3D coordinate system with origin at the top-left corner of the field is defined as follows:

- **X axis:** the positive axis points rightward from $x = 0$ to $x = 1200$ mm
- **Y axis:** the positive axis points downward from $y = 0$ to $y = 680$ mm

- **Z axis:** the positive axis points away from the camera into the scene. $Z = 0$ is taken as the playing field surface.

Three Z planes are used in the pipeline as follows:

- $Z = 0$ mm — the playing field surface
- $Z = -85$ mm — the plane where the rods are mounted with the player figurines, which is physically 85 mm above the field surface and therefore closer to the camera
- $Z = -118$ mm — the plane where the ArUco markers are placed at the four corners of the table, which is physically 118 mm above the field surface and therefore closer to the camera

Camera projection is a 3D to 2D operation and when the camera projects 3D world points onto an image, the Z coordinate directly affects where those world points land on the image plane. If the wrong Z is selected, the code will work without error but the points will land at the wrong location and produce incorrect results.

3.4.2 ArUco Marker Detection and Anchor Point Extraction

Four ArUco markers from dictionary `DICT_4X4_50` with IDs 0, 1, 2, 3 are used in this project to estimate the camera position and orientation. These markers are placed at the four corners of the table at a height of 118 mm above the field, as widely spread points give a better pose solution than clustered points.

The width of the marker border is 9.5 mm and therefore the world coordinates of the anchor points are at ± 9.5 mm offset from the field corners at a height of 118 mm above the field surface, i.e. at $Z = -118$ mm.

One corner per marker is selected and this is called the anchor point. These are the inner corners of each marker that face the centre of the field. These correspond to the known world coordinates used for pose estimation. After detection using `cv2.aruco` functions with default flags as discussed in section 2.3.2, the presence of all four marker IDs is verified before any operation such as pose estimation is carried out.

Figure 3.4 shows the four ArUco markers used in this project, with IDs zero to three from left to right. Each marker has a thick black border and a unique 4×4 binary pattern encoded inside.

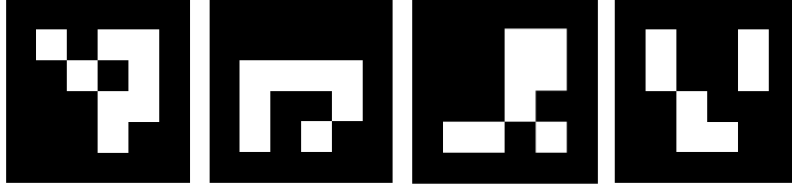


Figure 3.4: The four ArUco markers used in this project from DICT_4X4_50, with IDs zero, one, two, and three from left to right. Each marker has a thick black border surrounding a unique 4×4 binary pattern.

3.4.3 Pose Estimation and Homography Computation

The goal is to estimate the camera position and orientation relative to the world coordinate system and then use that to build a homography that maps the camera view to a top-down millimetre canvas.

Initial pose estimation: `cv2.solvePnP` solves this problem. Given the four known 3D world points, their corresponding 2D image projections, and K_{new} , the optimal camera matrix from the calibration stage, `rvec` and `tvec` are obtained that describe the camera pose in the 3D world. The `SOLVEPNP_ITERATIVE` flag uses a Levenberg–Marquardt style iterative method to refine the solution.

Non-linear refinement: The initial pose from `cv2.solvePnP` is used as the starting point and fed into `cv2.solvePnPRefineLM` to further refine the pose by performing non-linear Levenberg–Marquardt optimisation directly on the reprojection error, i.e. the sum of squared distances between the projected 3D points and the detected 2D points.

With the camera pose now known, the four field corner world points at $Z = 0$ are projected onto the image plane, producing four corresponding 2D pixel positions. `cv2.findHomography` then computes the 3×3 homography matrix that maps these four projected field corner pixels to the four corners of the target bird’s-eye canvas with padding.

The pose estimation and homography are computed once per video using a single frame at the beginning. The code searches up to the first 100 frames for one frame in which all four markers are visible. The resulting H , `rvec`, and `tvec` are then reused throughout that video.

The warped bird’s-eye view canvas has a size of $(1200 + 2 \times 50) \times (680 + 2 \times 50) = 1300 \times 780$ pixels, where the 50 mm padding is included on all sides.

3.5 Ball Color Calibration

3.5.1 Motivation for Per-Setup Calibration

The HSV bounds are determined once using the interactive color tuning tool and the resulting values are then hardcoded into the tracking pipeline for all subsequent recordings. A dedicated color tuning video is recorded before the actual game videos are recorded. The ball is moved intentionally either with the rods or by hand to all corners, edges, and the center of the field to capture all possible positions.

The video is then played in a window and with the help of the interactive tool, the video is paused whenever the ball is clearly visible. With the help of a mouse click, the color of the ball is picked and the HSV values of that pixel are captured, after which the video automatically resumes.

This is repeated four times to capture the HSV values of the ball at different positions on the field and under different lighting conditions. A single set of HSV values might misrepresent the ball color due to a particular light reflection or shadow at that position. By sampling four times across different ball positions on the field, the natural color of the ball is captured more reliably, making the detection more robust to lighting changes during game recordings.

3.5.2 HSV Range Computation

After the four clicks, the element-wise minimum and maximum across all three HSV values of the four samples are computed to create a lower and upper limit. Tolerance padding is then applied as follows:

- H : ± 10
- S : lower -40 , upper 255 .
- V : lower -50 , upper 255 .

Hue is quite stable and therefore only a small padding is needed. The reason for fixing the upper values of saturation and value to 255 is to capture the most vibrant colors and the brightest reflections on the ball surface.

The red wrap-around check: if $\min_h < 15$ and $\max_h > 165$, the standard min and max approach breaks as the range would traverse through the non-red part of the spectrum as well. Therefore a fixed wide range is used instead, with the lower limit set to 155 and the upper limit set to 25 , correctly bridging the $179-0$ boundary.

3.6 Ball Detection and Tracking

3.6.1 HSV Masking and Morphological Cleaning

After the warping process, the bird's-eye view image is converted from BGR to HSV using `cv2.cvtColor`. A binary mask is then created using `cv2.inRange` with the lower and upper red bounds extracted during the color tuning phase.

The morphological operations discussed in Section 2.5.3 are then applied with certain modifications. Since the number of iterations differs between the two operations, this is considered a modified version of morphological opening. Erosion is carried out with one iteration and dilation with two iterations. The reason for using one extra iteration of dilation is to merge fragmented blobs present in the scene, particularly the ball blob. Such fragmentation occurs when there is a specular reflection on the ball surface, causing the blob to split into disconnected parts.

`cv2.findContours` then finds all contours in the cleaned mask and the first contour whose radius is strictly between 5–35 pixels is accepted as the ball. Contours with a radius below five pixels are rejected as noise and those above 35 pixels are considered false positives from larger objects in the scene.

3.6.2 Kalman Filter for Robust Tracking

The Kalman filter, as discussed in Section 2.7, is used for robust tracking of the ball. The state vector tracks position and velocity in both axes and only position is measured directly from the detection.

For each frame, ball detection is performed first. The Kalman prediction step is then called regardless of whether the ball is detected. If the ball is detected, the correction step is triggered and the filter updates its state using the measured position. If the ball is not detected, the correction step is skipped and the predicted position is used directly as the ball position, with the status logged as `Occluded`.

The `processNoiseCov` matrix is set to $0.03 \times I$, where I is the 4×4 identity matrix. This small value reflects high confidence in the constant velocity model, producing smooth ball movement estimates. The `measurementNoiseCov` is not explicitly defined and the OpenCV default identity matrix is used. The trade-off of this small α is a lag of two to three frames before the filter fully catches up with a new direction after a sudden hit. This is acceptable as smooth tracking is more important than capturing sudden movements, since the hit detection logic operates on entry and exit zones rather than instantaneous velocity.

3.6.3 Ball Status State Machine

After the Kalman filter predicts the position for frames where detection is missed, all frames now have a ball position that is either directly measured or predicted. Based on these positions, each frame is classified into one of five mutually exclusive states.

S1 — On Field: the ball is detected and its position after the radius check lies within the field bounds.

S2 — Wall: the ball is detected outside the field bounds and not near either goal mouth.

S3 — Goal: the ball is near the goal mouth for more than 5 consecutive frames. A goal counter is used to track and confirm this.

S4 — Occluded: no ball is detected in the current frame, however the Kalman filter prediction falls within the field bounds and is used as the ball position.

S5 — Searching: no ball is detected and the Kalman filter prediction also falls outside the field bounds. No position is available.

The goal counter is incremented in the following two cases:

1. The ball is detected outside the field but near the goal mouth, within 100 mm of the goal Y centre on either side
2. The last known on-field position of the ball is within 50 mm of the left or right edge and the ball is not detected in the current frame

If a goal is detected on the left side of the field, i.e. $X < 50$ mm, then White scored. If a goal is detected on the right side, i.e. $X > 1150$ mm, then Black scored. Also, each goal is registered only once by checking that the previous frame status was not already goal and thereby ensuring that consecutive goal frames do not result in multiple counts.

3.7 Player Position Detection

3.7.1 Rod Line Projection into the Warped Image

Before determining the player positions, the rods must be located in the warped bird's-eye image. These rods are physical 3D objects at known world positions and can therefore be projected mathematically. For each of the eight rods, 200 equally spaced 3D points are generated along the rod at $Z = -85$ mm, between $Y = 0$ and $Y = 680$ mm at the rod's fixed X position. These 200 points are then projected into the undistorted image using `cv2.projectPoints` with the rotation vector and translation vector recovered from the first frame. Each projected point is then mapped onto the warped image using the homography. In total, $200 \times 8 = 1600$ points are projected once per video and reused throughout to save computation.

3.7.2 Intensity Profile Extraction and Filtering

Once the 200 projected points are available for each rod, the brightness at each point is sampled to build a 1D brightness signal along the rod. The BGR pixel value at each of the 200 points is converted to grayscale luminosity using the standard perceptual weighting formula, producing a 200-element 1D array. The reason for using grayscale is that the players are either dark or light-colored and the detection relies solely on brightness contrast. Using three color channels would also work but would add unnecessary complexity.

The raw signal is noisy due to lighting variations, specular reflections and shadows, and therefore needs to be filtered. The signal is reshaped to $(1, 200)$ to make it compatible with `cv2.bilateralFilter`, which expects a 2D input. It is then filtered and flattened back to a 1D array. The parameters used are $d = 9$, $\sigma_{\text{color}} = 30$ and $\sigma_{\text{space}} = 100$.

3.7.3 Peak Detection, Region Merging, and Stopper Identification

This step is the core of the player position detection. The goal is to find the Y positions of the players on each rod. The raw signal has low values at dark pixels and high values at bright pixels. Inverting the signal makes dark figurines form bright peaks, making detection more intuitive since maxima are searched rather than minima. An adaptive threshold is computed as the 50th percentile of the inverted signal plus an offset of 30, and all regions above this threshold are accepted.

All such regions above this threshold are identified and each region is described by its start position, end position, centre, width and peak height.

Adjacent regions are merged if the gap between them is less than or equal to 10 mm and the first region's width is below 45 mm. This handles specular reflections which can split a single figurine into two separate peaks.

From all merged regions, the two regions with the highest peak intensity that meet the minimum width threshold are identified as the physical end-stoppers. The minimum width threshold differs by team: 35 mm for dark Black figurines and 10 mm for lighter White figurines. Each player position is then computed relative to these stoppers.

Since rods can be pushed from the top to the bottom of the field, the push direction is determined by checking which stopper is closer to which edge of the table. The inner edge of the stopper closer to the nearest table boundary is taken as the point of interest (POI). All player position offsets are hardcoded per rod and direction relative to the POI, and applied to compute the final player Y positions. These offsets along with other rod and player information are stored in a dictionary declared at the beginning of the code. For goalkeeper rods, which have only one figurine, the player position is taken as the midpoint between the two stoppers directly without applying any offset. This is because

the goalkeeper figurine sits in a fixed central position between the two stoppers and no offset is needed.

3.7.4 Smart Stabilisation

Even after applying the bilateral filter and region merging, there is noticeable fluctuation in the detected player positions from frame to frame due to noise, lighting changes and subpixel variations. These fluctuations do not represent real physical rod movement. For each rod, the newly detected player positions are compared to the previously accepted positions and the maximum displacement across all players on that rod is computed. If the maximum displacement is less than or equal to 3 mm, the previous positions are kept unchanged. New positions are accepted only if at least one player has moved by more than 3 mm. When a rod is physically moved during gameplay, the movement is on the order of tens of millimetres, so the 3 mm threshold sits cleanly between detection noise and real movement.

3.8 Hit Detection — Branch 1

3.8.1 Hit Zone Definition and Ball-in-Zone Check

Each player on the rod has a particular hit zone in which the player can physically hit the ball. These hit zones are centered on the player's current position in the warped millimetre coordinate system. When the rod, which sits on a fixed plane at $Z = -85$ mm, rotates, the player figurines placed on the rod rotate as well and this sweeping effect laterally in both directions forms the width of the hit zone. Figure 3.5 illustrates this geometry. The large circle represents the full sweep diameter of the figurine foot during rod rotation. The two smaller circles represent the ball at the extreme positions of tangential contact on either side. The resulting bounding box gives an estimate of the region in which ball-player contact can physically occur. The height of the hit zone is determined by the width of the foot and the ball diameter with a certain padding, considering the error in ball detection and the error in stopper detection. After taking into account this geometry, the hit zone dimensions are defined as follows. The width is 120 mm and the height is 80 mm.

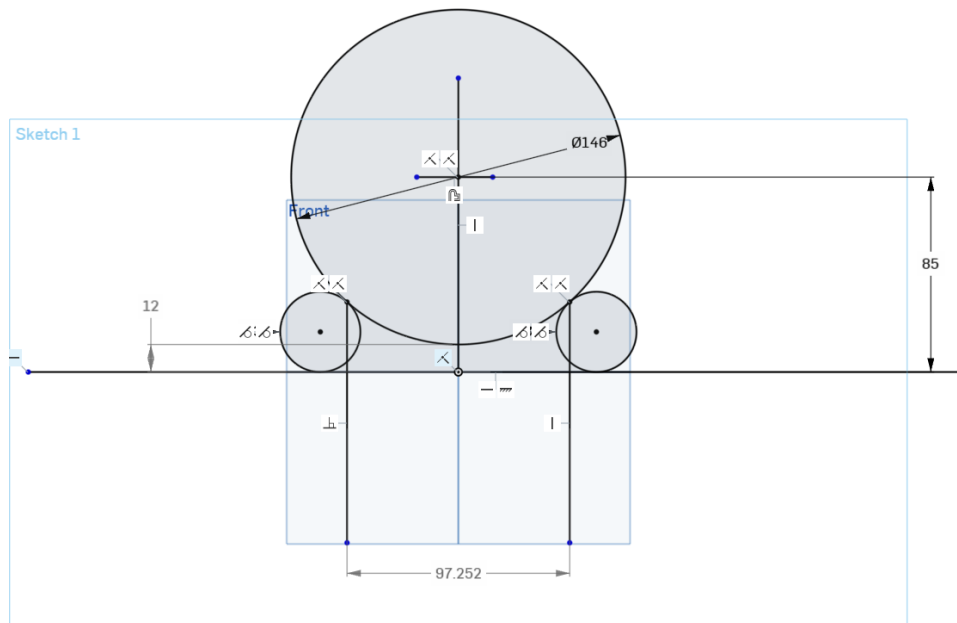


Figure 3.5: Geometric derivation of the hit zone dimensions. The large circle represents the sweep diameter of the figurine foot during full rod rotation. The two smaller circles represent the ball at the extreme positions of tangential contact. The resulting bounding box provides the basis for the hit zone width of 120 mm and height of 80 mm used in the implementation, which include an additional padding margin.

A simple axis-aligned bounding box test known as AABB is performed for each frame. This test is run for all 22 players simultaneously. The AABB test is the cheapest possible spatial containment test. It considers four parameters per player and is therefore computed 22 times per frame. These four parameters are the X position of the rod, the Y position of the player, and the X and Y positions of the ball. If no ball position is available, no hit is detected.

3.8.2 Entry/Exit State Machine Per Player

A single Boolean that says whether the ball is in the hit zone or not is not enough to classify a hit. Information such as how the ball entered the zone, how long it stayed, and how it left are crucial to explain the type of hit or interaction that took place. This is what the state machine tracks. For each player, a dictionary is maintained with the following fields:

- `active` — whether the ball is currently inside this player's zone
- `entry_vx`, `entry_vy` — ball velocity components at the moment of entry
- `entry_speed` — ball speed magnitude at the moment of entry
- `entry_frame` — frame number at which the ball entered the zone
- `frames_inside` — number of frames the ball has been inside the zone
- `min_speed_inside` — minimum speed observed while the ball was inside the zone
- `was_stopped` — whether the ball came to rest inside the zone at any point during the contact

Note that the exit velocity is not stored in the dictionary. Since the classification logic runs at the exit frame, the current ball velocity at that moment is directly available and does not need to be tracked across frames.

There are three transition events that can take place.

Entry (`active` = `False` → `True`): the current ball velocity components `entry_vx`, `entry_vy` and `entry_speed` are recorded. `frames_inside` is set to 1, `min_speed_inside` is initialised to the current speed, and `was_stopped` is set to `True` if the ball is already moving slower than 0.10 m/s at entry, otherwise `False`.

Stay (`active` = `True`, ball still inside): `frames_inside` is incremented by 1. If the current speed is lower than `min_speed_inside`, it is updated. If the current speed drops below 0.10 m/s, `was_stopped` is set to `True` and remains `True` for the rest of this contact window.

Exit (`active` = `True` → `False`): the full contact window is now available. All of the tracked parameters are passed to the classification logic to determine which type of hit took place, after which `active` is set to `False`.

A simpler approach would be to classify a hit on every frame the ball is inside the zone. However this simple approach fails to capture the type of hit. The entry/exit logic evaluates the entire contact window, making it robust to both missed detections and detection errors within the window.

3.8.3 Hit Classification Logic

The entry/exit logic runs at the exit frame. This logic has two main branches depending on how long the ball was inside the hit zone, and several sub-branches depending on speed and direction.

Branch 1 is triggered when `frames_inside` < 3, defined as a quick contact. For a video at 30 frames per second, this is less than approximately 67 ms. This is too fast for any player to control the ball and the classification is therefore purely based on the direction change. The exit velocity is taken as the current ball velocity at the moment of exit and is

not stored in the state dictionary since it is always available as the current frame velocity when the exit event triggers. The angle between the entry and exit velocity vectors is computed using the dot product:

$$\theta = \arccos \left(\frac{\vec{v}_{entry} \cdot \vec{v}_{exit}}{\|\vec{v}_{entry}\| \cdot \|\vec{v}_{exit}\|} \right) \quad (3.1)$$

The output is always in degrees. There are three outcomes:

- $\theta > 150^\circ$ — classified as a **Block**, because the player stopped and reversed the direction of the ball
- $25^\circ \leq \theta \leq 150^\circ$ — classified as a **Brush**, because the ball was deflected at an angle as it passed through the zone
- $\theta < 25^\circ$ — classified as **no hit**, because the ball passed through with almost no deflection. The 25° threshold compensates for errors in ball detection

Branch 2 is triggered when `frames_inside` ≥ 3 , meaning the ball stayed inside the zone for at least 100 ms. This indicates a controlled contact with the player figurine. This branch has two sub-branches.

Sub-branch A is triggered when `was_stopped` is True. This physically means the ball came to rest and the player controlled the ball. The outcome is purely dependent on the exit speed:

- Exit speed ≥ 0.50 m/s — **Strong Kick**
- Exit speed ≥ 0.30 m/s — **Weak Kick**
- Exit speed < 0.30 m/s — **Stop (No Kick)**, which could also mean the player stopped the ball and it slowly rolled out

Sub-branch B is triggered when `was_stopped` is False. This physically means the ball kept moving through but the player maintained contact for multiple frames. In the context of foosball this means the ball was dribbled. The classification is purely dependent on the exit speed with the same thresholds as Sub-branch A:

- Exit speed ≥ 0.50 m/s — **Strong Dribble & Kick**
- Exit speed ≥ 0.30 m/s — **Weak Dribble & Kick**
- Exit speed < 0.30 m/s — **Dribble (No Kick)**, meaning the player just redirected the ball gently

These eight hit types are grouped into five categories in the output CSV: Kick, Dribble, Block, Brush, and Stop. Strong Kick and Weak Kick map to Kick, Strong Dribble & Kick and Weak Dribble & Kick map to Dribble, and Block, Brush, and Stop (No Kick) map to their respective categories directly.

3.9 Feature Extraction — Branch 2

3.9.1 Intentional Simplification Relative to Branch 1

Branch two of the feature extraction for the learning pipeline follows the same processes as Branch 1. The main difference is that hit detection is not considered here. No entry/exit state machine, hit categories, or hit logging are stored in the CSV file. The reason for this is that whenever hit detection is included, the computational cost per frame increases drastically i.e. 22 hit zones must be checked per frame along with maintaining the state for each player in each frame and classifying the exits. This is all unnecessary additional work for the learning pipeline as the model will learn to find meaning in the available positional data. This makes Branch 2 faster and the CSV schema cleaner than Branch 1.

3.9.2 The 25 Feature Columns

A total of 25 features are extracted from the data and these 25 features are used for learning. This includes the x and y position of the ball, normalized by 1200 and 680 respectively, giving a range of $[0, 1]$. The states that are neither On Field nor Occluded are considered unreliable. In such cases, the ball position is set to 0.5 as a fallback. This value represents the centre of the field and is the least informative value in the data.

The 22 rod player Y positions across the 8 rods are also normalised by 680, ranging from 0 to 1. These are stored in columns `y_rod_1_p1` through `y_rod_8_p1`, with each rod having as many player columns as it has figurines i.e. 1 for goalkeepers, two for defenders, 3 for attackers, and 5 for midfielders. Whenever a player is not detected, a fallback value of -1 is assigned. The reason for using -1 is that it lies outside the valid normalization range of $[0, 1]$ and therefore the model learns over time that this indicates a case of player not detected.

The last feature included in the 25 features is `frame_norm`. This is the frame number divided by the total number of frames in the video, i.e. `frame_number / 1800`. This acts as a global temporal position indicator and is recomputed once the sequencing is done.

3.10 Post-Processing and Dataset Construction

3.10.1 Frame Filtering

As discussed in the previous section, whenever the status is neither On Field nor Occluded, the ball position is unreliable. Therefore such rows are dropped from the CSV file. This includes rows where the status is Wall, Goal, or Searching. When the status is On Field, the ball is accurately detected by the camera and the position is reliable. When it is

Occluded, the ball position is estimated by the Kalman filter and this is also reliable. For all other states the ball position is either lost, not found, or set to the fallback value of 0.5 and hence is not reliable. Including these unreliable states would be meaningless as it could mislead the model into learning uninformative positions.

This filter also implicitly removes the dead-ball time. The dead-ball time is defined as the time between a goal and the next kickoff. This could also include the time before the ball enters the playing area at the beginning of the game. The main purpose of frame filtering is to include only active game-play and to ensure that the sequence classifier receives only active game-play frames.

3.10.2 Temporal Sequencing and Local Frame Norm

Once a total clean frame count with reliable ball positions is obtained, these frames are divided into six sequences. The floor of this division is taken as the sequence length. The remaining frames, a maximum of five, are dropped at the end. By doing this, six consecutive non-overlapping segments are available for further analysis.

The reason for using six sequences is that the dataset size increases by a factor of six i.e. from 57 videos, 342 sequences are obtained in total across the train and test datasets, of which 294 are training sequences and 48 are test sequences. Each sequence has around 200 to 230 frames, which is enough to capture meaningful gameplay patterns. Whenever a higher number of sequences is selected, the duration of gameplay in each sequence reduces drastically and this would not be helpful for the model to learn any patterns. As discussed before, `frame_norm` is recomputed after the division. Each segment acts as an independent sequence and therefore a local `frame_norm` is calculated, ranging from 0.0 to 1.0. Each sequence therefore has its own encoded temporal positioning representing a window of approximately 7 seconds of active gameplay.

3.10.3 Zero-Padding and Final Dataset Arrays

After filtering and sequencing, the sequence lengths vary across videos because of differences in the number of clean frames available per video. Therefore all sequences are padded with zeros at the end to match the length of the longest sequence in the training set. For the test set, sequences are either padded or truncated to match the same length.

The class weights are computed using the inverse frequency weighting formula described in Section 2.9.6 and passed to `CrossEntropyLoss` during training.

3.11 TCN Model Architecture and Training

3.11.1 Input Representation and Dataset Class

The model takes $\mathbf{X}$ as a NumPy array of shape $(N, \text{seq_len}, 25)$ and $\mathbf{y}$ as integer labels. These are converted into PyTorch tensors. The sequence tensor is then permuted from $(\text{seq_len}, 25)$ to $(25, \text{seq_len})$ to match the input convention expected by `Conv1d`, where the number of channels corresponds to the 25 features and the time dimension corresponds to the sequence frames. Shuffling is enabled in the `DataLoader` during training, ensuring that the model sees the sequences in a random order so that it does not memorise patterns. With `drop_last=True`, the batch size is ensured to be exactly 16 during training. Both shuffling and `drop_last` are set to `False` during testing.

3.11.2 TCN Block Architecture

The Temporal Convolutional Network architecture used has two dilated `Conv1d` layers in sequence. Each layer is followed by `BatchNorm1d`, `ReLU` activation, and `Dropout(0.4)`. Padding is set to $(\text{kernel_size} - 1) \times \text{dilation}$ to maintain the output length equal to the input length. After each convolution, the output is trimmed to the input length, enforcing causal behaviour.

Whenever the input channels are not equal to the output channels, which is observed in the first block only, i.e. from 25 to 16 channels, a 1×1 convolution projects the input to 16 channels before the residual addition. When the channels are equal, an identity connection is used. The final output is $\text{ReLU}(\text{main path} + \text{residual})$.

Three TCN blocks with dilations of one, two, and four are used with a kernel size of three. Figure 3.6 illustrates the full architecture and the internal structure of one block. When these three blocks are stacked together, the total receptive field becomes 15, computed as:

$$\text{RF} = 1 + \sum_{i=0}^{L-1} (k - 1) \cdot d_i = 1 + (3 - 1) \cdot (1 + 2 + 4) = 15 \quad (3.2)$$

With sequences of around 200 frames, this covers a meaningful window of temporal context. After the three blocks, the output has shape $(\text{batch}, 16, \text{seq_len})$. Global average pooling takes the mean across the sequence length dimension, producing a fixed-size vector of shape $(\text{batch}, 16)$. This vector is then fed to a linear classifier that outputs three class scores.

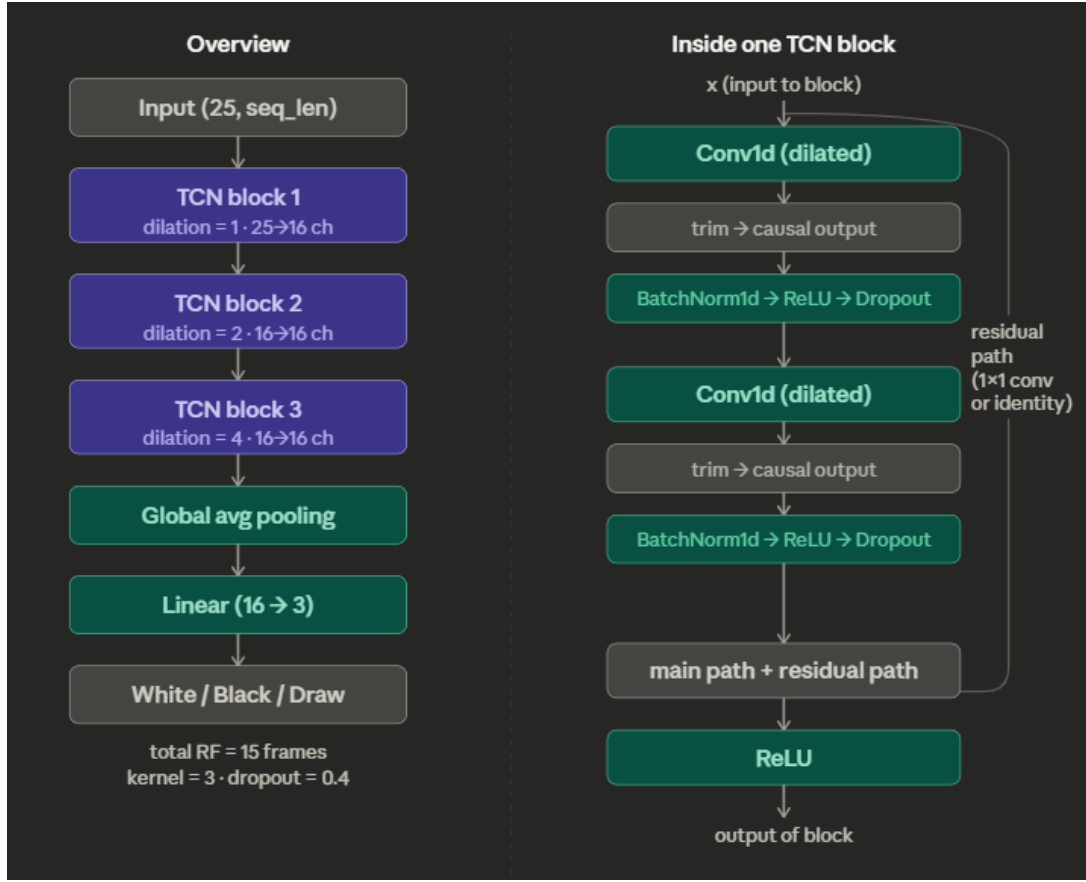


Figure 3.6: TCN architecture used in this work. Left: overview of the full pipeline from input to output. Right: detailed view of one TCN block showing the two dilated causal convolution layers, causal trimming, batch normalisation, ReLU, dropout, and the residual connection. In Block 1 the residual path uses a 1×1 convolution to project from 25 to 16 channels. In Blocks 2 and 3 the residual path is an identity connection since the channel dimensions are equal. Figure generated using Claude AI as a visualisation aid.

3.11.3 Training Procedure and History Logging

The training loop per epoch consists of a forward pass followed by the weighted cross-entropy loss, a backward pass via `loss.backward()`, and an Adam optimizer step with a learning rate of 1×10^{-3} . This is repeated for each batch. After each epoch, a validation phase is carried out with `model.eval()` and `torch.no_grad()`, performing only a forward pass on the test loader.

The `ReduceLROnPlateau` scheduler steps on the validation loss with a patience of five and a factor of 0.5. This means that whenever the validation loss does not improve for five consecutive epochs, the learning rate is reduced by half.

The following values are logged per epoch: training loss, validation loss, training accuracy,

and validation accuracy. The best model is saved whenever the validation accuracy improves. This checkpoint is loaded for all subsequent testing and evaluation.

Note that in this work the test set is used as the validation set, as the dataset is too small to reserve a separate validation split without further reducing the training data.

4 Experiments

4.1 Implementation Details

4.1.1 Experimental Environment and Model Hyperparameters

The experiments are carried out on a laptop equipped with a 12th Gen Intel Core i5-1240P processor and a 15.7 GB of RAM. Since the dataset is small and lightweight, no dedicated GPU is required and all training is done on the CPU. Table 4.1 summarises the hyperparameters used for all experiments.

Table 4.1: TCN model hyperparameters used for all experiments.

Parameter	Value
Batch size	16
Epochs	20
Learning rate	1×10^{-3}
Optimizer	Adam
Kernel size	3
TCN blocks	3
Channels per block	16
Dropout	0.4
Dilation schedule	1, 2, 4
LR scheduler	ReduceLROnPlateau
Scheduler patience	5
Scheduler factor	0.5
Evaluation seeds	42, 7, 123, 99, 21

4.2 Calibration Results

4.2.1 Corner Detection Quality

Figure 4.1 shows the detected inner corners overlaid on one calibration frame, confirming that the corner detection is visually accurate across the board. The board is held at an angle, which confirms that the video recorded for calibration contains frames at different positions and orientations.

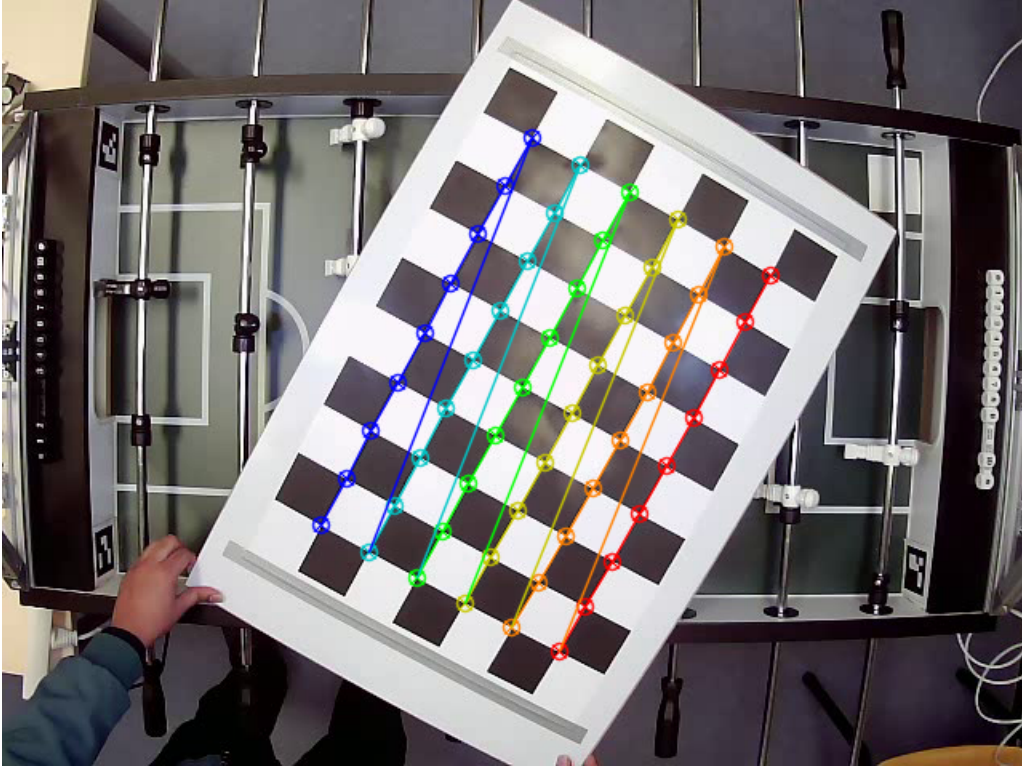


Figure 4.1: Detected inner corners of the checkerboard overlaid on a calibration frame. The coloured dots represent the detected corner points and the coloured lines connect them row by row.

4.2.2 Reprojection Error and Intrinsic Parameters

After applying the three quality filters described in Section 3.2.2, 40 images out of 5000 recorded frames were accepted and used for calibration. The board sizes across the accepted images ranged from 198 to 242 pixels, confirming that frames at different distances from the camera are included. The two worst images are then removed during iterative refinement, leaving 38 images for the final calibration solve.

RMS Reprojection Error: The initial RMS reprojection error is found to be 0.1608 pixels, which is excellent in itself, and by using the worst-image removal, the RMS reprojection error is further reduced to 0.1585 pixels. The improvement confirms the effectiveness of the worst-image removal. The reduced error is classified as excellent as it is well below the defined threshold of 0.5 pixels.

Intrinsic Matrix K :

$$K = \begin{bmatrix} 599.5 & 0 & 301.9 \\ 0 & 599.7 & 257.6 \\ 0 & 0 & 1 \end{bmatrix} \quad (4.1)$$

The values of f_x and f_y are found to be 599.5 and 599.7 pixels respectively. These are very close to each other, suggesting that the pixels are very close to square. c_x and c_y are found to be 301.9 and 257.6, which are 18.1 and 17.6 pixels away from the expected image centre of (320.0, 240.0) respectively. These values are well within the set tolerance of 30 pixels. c_x is shifted to the left and c_y is slightly shifted downward, which can be attributed to manufacturing imperfections.

Distortion Coefficients D :

$$D = [-0.352, 0.130, 0.000349, -0.000471, 0.013] \quad (4.2)$$

The distortion coefficients extracted from D are: $k_1 = -0.352$, $k_2 = 0.130$, $k_3 = 0.013$, $p_1 = 0.000349$, $p_2 = -0.000471$. The k_1 value indicates dominant radial distortion and the negative sign confirms barrel distortion. k_2 represents the second order radial correction and k_3 shows a small higher order radial correction. The values of p_1 and p_2 indicate very small tangential distortion, which is nearly negligible.

Optimal Camera Matrix K_{new} :

Using K and D with **alpha=0.0**, the optimal camera matrix K_{new} is computed. This is the matrix actually used throughout the detection and tracking pipeline for pose estimation and point projection:

$$K_{new} = \begin{bmatrix} 533.7 & 0 & 297.0 \\ 0 & 563.3 & 260.3 \\ 0 & 0 & 1 \end{bmatrix} \quad (4.3)$$

Comparing K_{new} with K , it can be observed that f_x has decreased from 599.5 to 533.7 and f_y has decreased from 599.7 to 563.3. The principal point c_x has shifted from 301.9 to 297.0 and c_y from 257.6 to 260.3. These changes are consequences of **alpha=0.0**, which crops the Black border pixels that appear after undistortion, effectively adjusting the camera matrix to match the valid pixel region of the undistorted image.

4.2.3 Undistortion Quality

This subsection shows the comparison of the original distorted image and the undistorted image.

The following observations can be made from the original distorted image, shown in Figure 4.2:

- The checkerboard edges appear slightly curved, which is most evident near the corners of the board.
- The presence of barrel distortion is clearly visible.
- The table edges in the background also show a slight curvature.

The following observations can be made from the undistorted image, shown in Figure 4.3:

- The checkerboard edges appear straighter.
- The board corners appear more rectangular.
- The table edges in the background also appear slightly straighter.
- The image is very slightly cropped at the edges due to $\alpha=0.0$.

Although the distortion is mild, it is consistent with the obtained distortion coefficient $k_1 = -0.352$, which is the dominant coefficient despite being modest in magnitude.

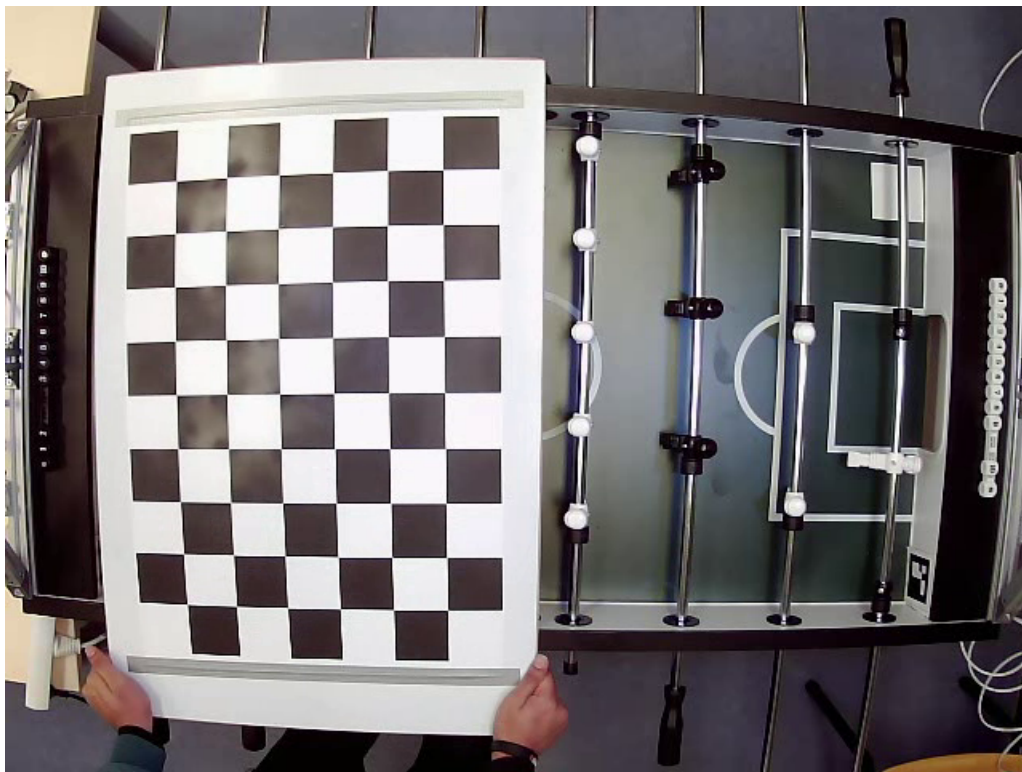


Figure 4.2: Original distorted frame showing visible barrel distortion. The checkerboard edges appear slightly curved, particularly near the corners, and the table edges in the background also show a slight curvature.

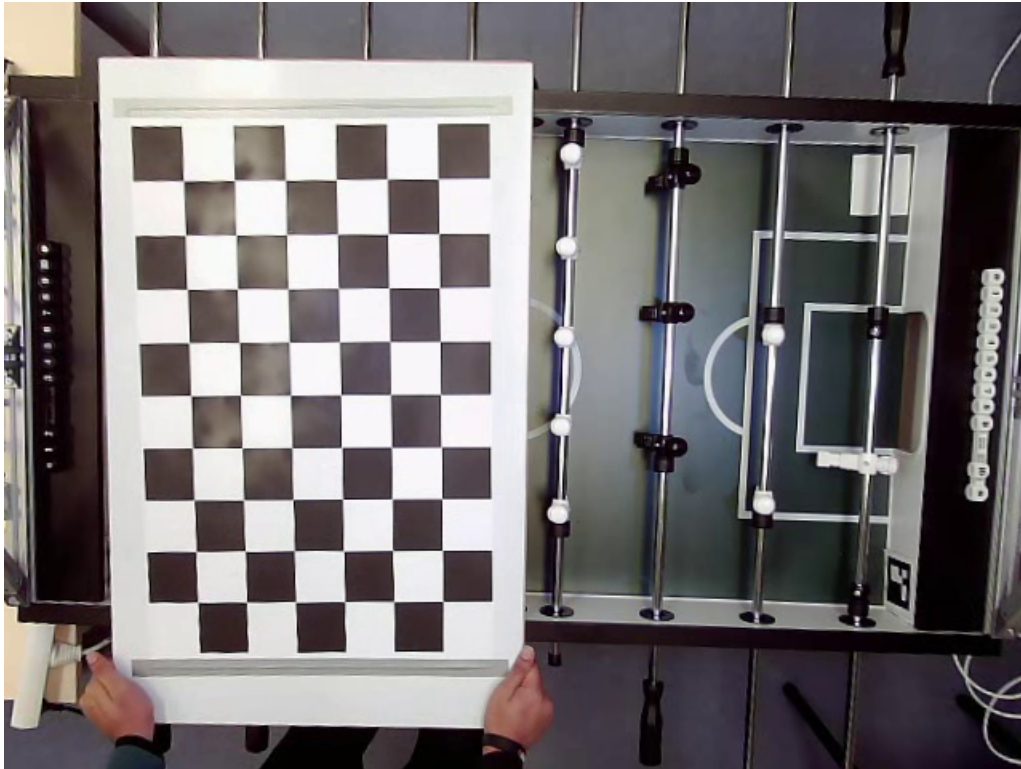


Figure 4.3: Undistorted frame after applying the calibration maps `map1` and `map2`. The checkerboard edges now appear straight, the board corners are more rectangular, and the slight curvature visible in the original image has been corrected. The image is very slightly cropped at the edges due to `alpha=0.0`.

4.2.4 Dataset Statistics

Each of the 57 recorded videos is split into six sequences during post-processing, thereby obtaining $57 \times 6 = 342$ sequences in total.

Out of the 57 recorded videos, 49 are used for training and 8 are held out for testing. The class distribution of the full dataset is as follows:

- White: 30
- Black: 11
- Draw: 16

The test set contains:

- White: 3
- Black: 3
- Draw: 2

The class imbalance and how it is handled is discussed in Section 2.9.6.

4.3 Perspective Correction Results

4.3.1 Homography Quality and Bird's-Eye View

The homography matrix H obtained from the perspective correction for one representative video is as follows:

$$H = \begin{bmatrix} 2.467 & 0.004 & -144.502 \\ -0.063 & 2.285 & -152.576 \\ 0.000 & 0.000 & 1.000 \end{bmatrix} \quad (4.4)$$

Using this matrix, the warped bird's-eye view is generated as discussed in the section 3.4. Figure 4.4 shows the detected and projected points on the undistorted image. The green dots represent the anchor points detected on the ArUco markers at $Z = -118$ mm. The yellow dots are the offset anchor points which lie on the border of the platform and are physically directly above the field corners and are also at $Z = -118$ mm. The red dots are the projected field corner points at $Z = 0$ mm, i.e. the actual corners of the playing field. The red lines drawn from the yellow points to the red points indicate the projection path from the platform plane to the field surface plane.

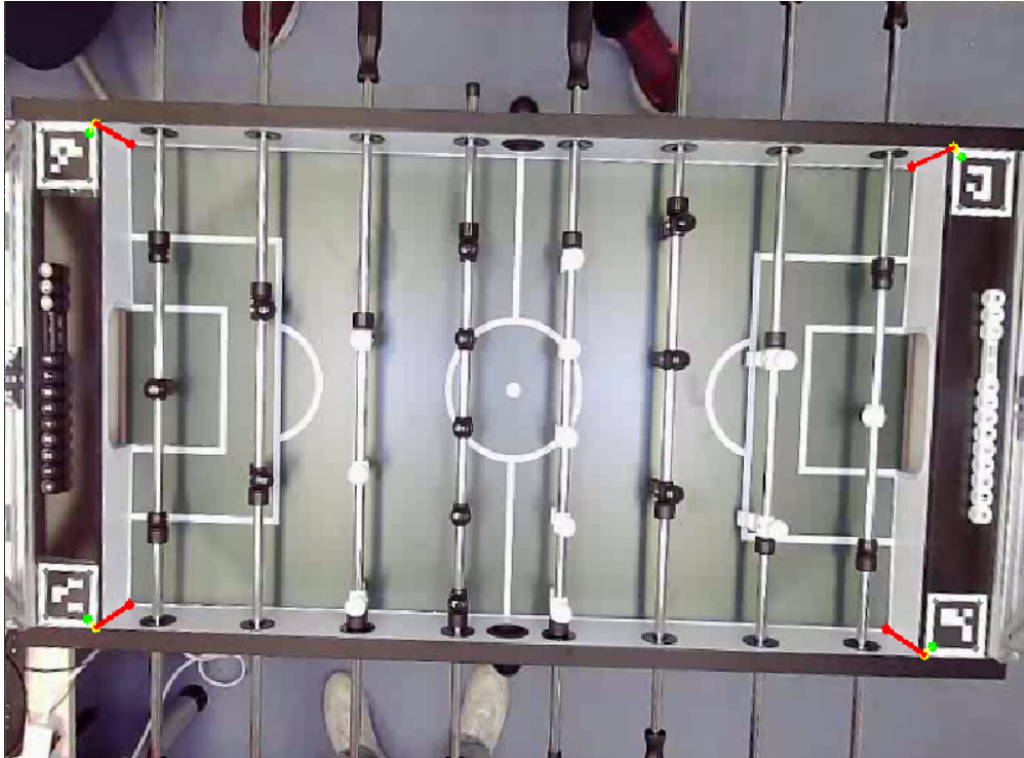


Figure 4.4: Detected ArUco anchor points (green), offset border points (yellow), and projected field corner points (red) on the undistorted image. The red lines indicate the projection from $Z = -118$ mm to $Z = 0$ mm.

Figure 4.5 shows the warped bird's-eye view of the playing field. A blue rectangle is overlaid on the image and all four corners of the rectangle coincide exactly with the four corners of the playing field, confirming that the homography maps the field correctly onto the canvas.

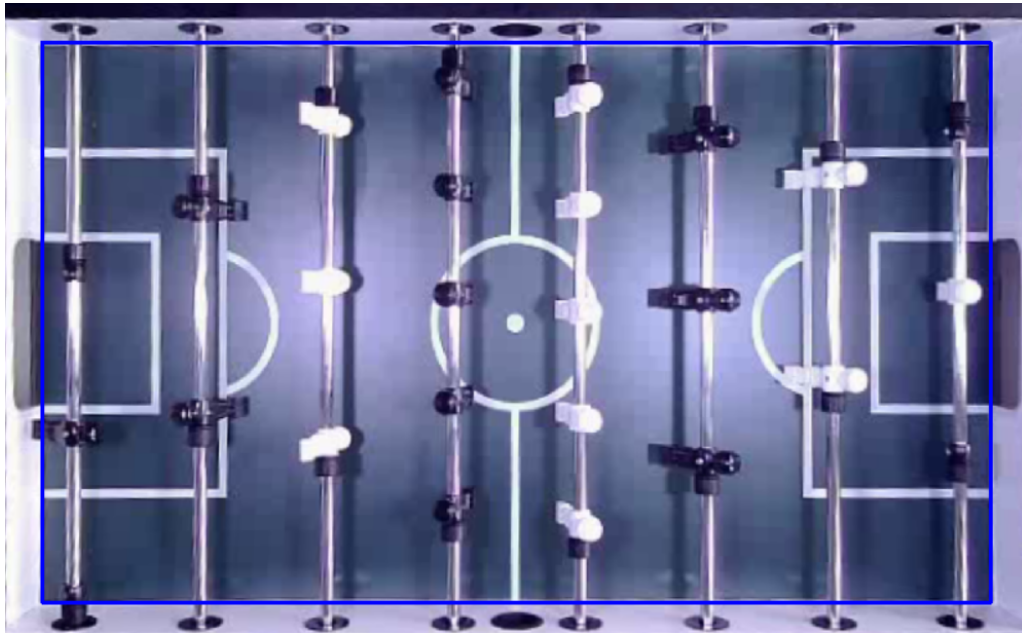


Figure 4.5: Warped bird's-eye view of the playing field. The blue rectangle overlaid on the image confirms that all four field corners are correctly mapped to the canvas corners.

4.3.2 Rod Line Projection Accuracy

Figure 4.6 shows the projected rod lines overlaid on the warped bird's-eye image. The Black team rods are represented by red lines and the White team rods are represented by cyan lines. These rod lines align visually with the physical rods in the image, confirming that the projection at $Z = -85$ mm, i.e. 85 mm above the field surface and closer to the camera, gives the correct position of the rods in the warped view. The rods are labelled R1 to R8 from left to right.

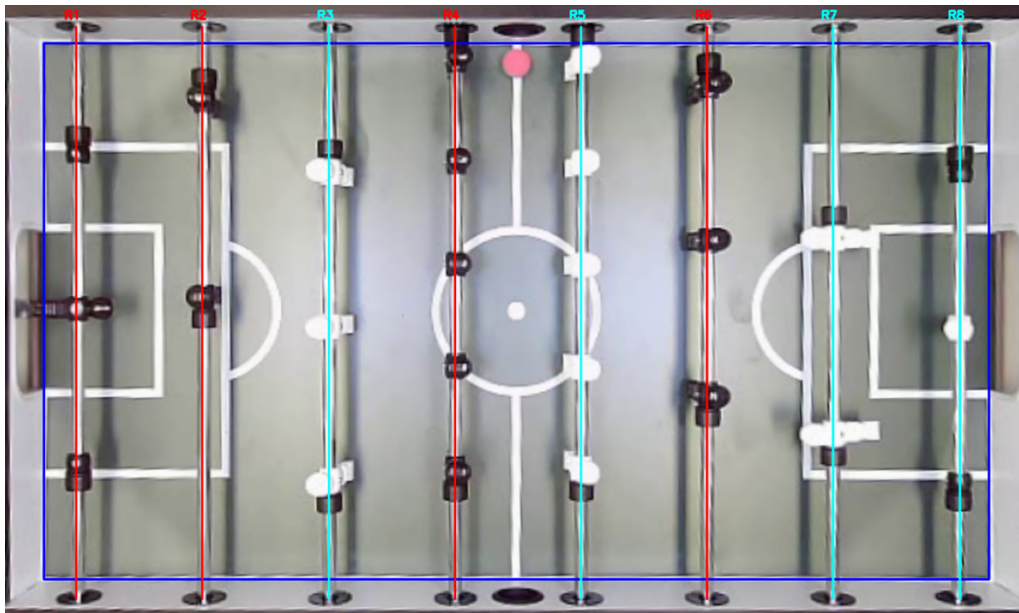


Figure 4.6: Projected rod lines overlaid on the warped bird's-eye image. Red lines represent Black team rods and cyan lines represent White team rods, labelled R1 through R8 from left to right.

4.4 Ball Detection and Tracking Results

4.4.1 HSV Masking Result

Figure 4.7 shows the binary mask generated by the colour tuning tool after sampling the ball HSV values at four locations on the field. All red regions in the scene are detected as white blobs in the mask. A false positive is clearly visible as a large white blob, which corresponds to an electronic component hanging from the aluminium frame that is caught in the camera's field of view. False positives are rejected by the radius filter described in Section 3.6.

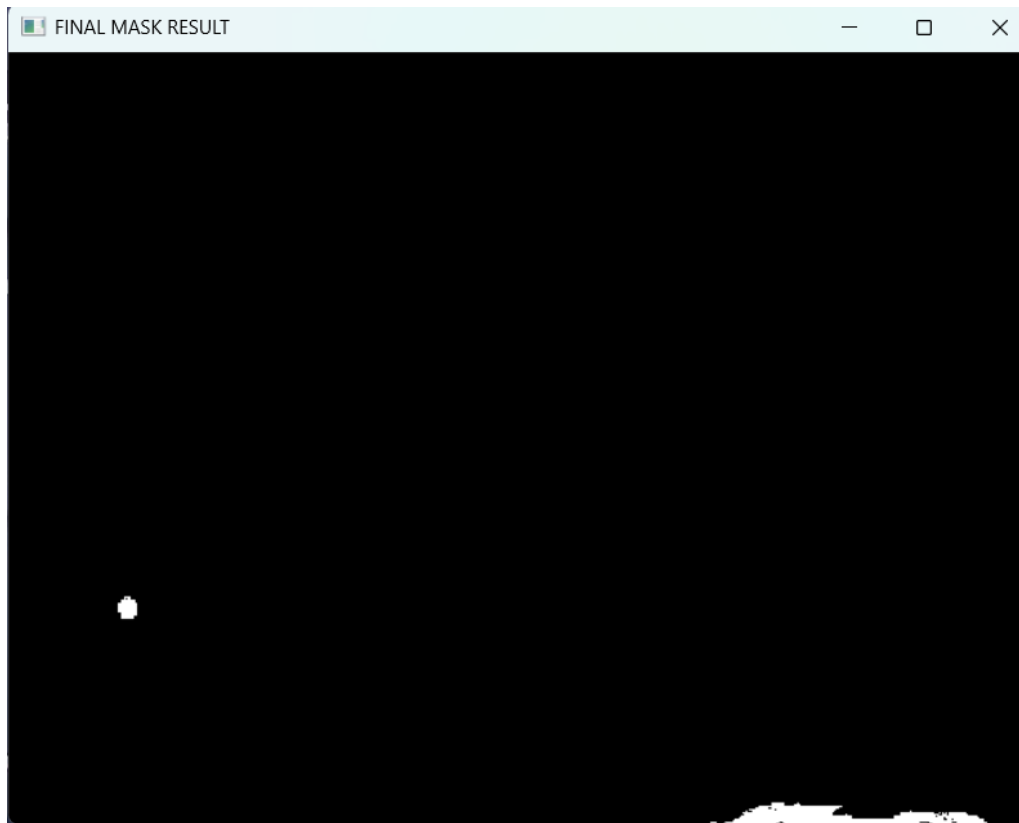


Figure 4.7: Binary mask generated after HSV thresholding. The small white blob is the ball and the large white blob at the bottom right is a false positive from an electronic component on the aluminium frame.

4.4.2 Detection Rate and Status Distribution

Table 4.2 shows the ball status distribution across the full dataset of 57 videos.

Table 4.2: Ball status distribution across the full dataset.

Status	Frames	Percentage
On Field	75,547	73.63%
Goal	13,618	13.27%
Searching	8,336	8.12%
Occluded	4,099	4.00%
Wall	1,000	0.97%
Total	102,600	100.00%

Table 4.2 shows that On Field status accounts for 73.63% of all frames, i.e. 75,547 frames, which means the ball is directly detected most of the time. The Occluded status accounts for 4.00%, i.e. 4,099 frames, where the Kalman filter stepped in and predicted the ball position in the absence of a direct detection. The Searching status accounts for 8.12%, i.e.

8,336 frames, which is a combination of frames before the game begins and frames where the ball is completely lost. The Goal status is high at 13.27%, i.e. 13,618 frames. This is expected and is justified by the goal confirmation logic. Once a goal is detected, the status remains goal as long as the ball stays near the goal area thereby producing a high frame count per goal event. The Wall status is the lowest at 0.97%, which is expected as wall contact events are infrequent.

4.5 Player Position Detection Results

4.5.1 Intensity Profile and Color Strip

Figure 4.8 shows the actual pixel values sampled along the rod line for all 8 rods. Two types of bands are visible: dark bands representing the Black figurines and stoppers, and bright bands representing the White figurines and the rod itself.

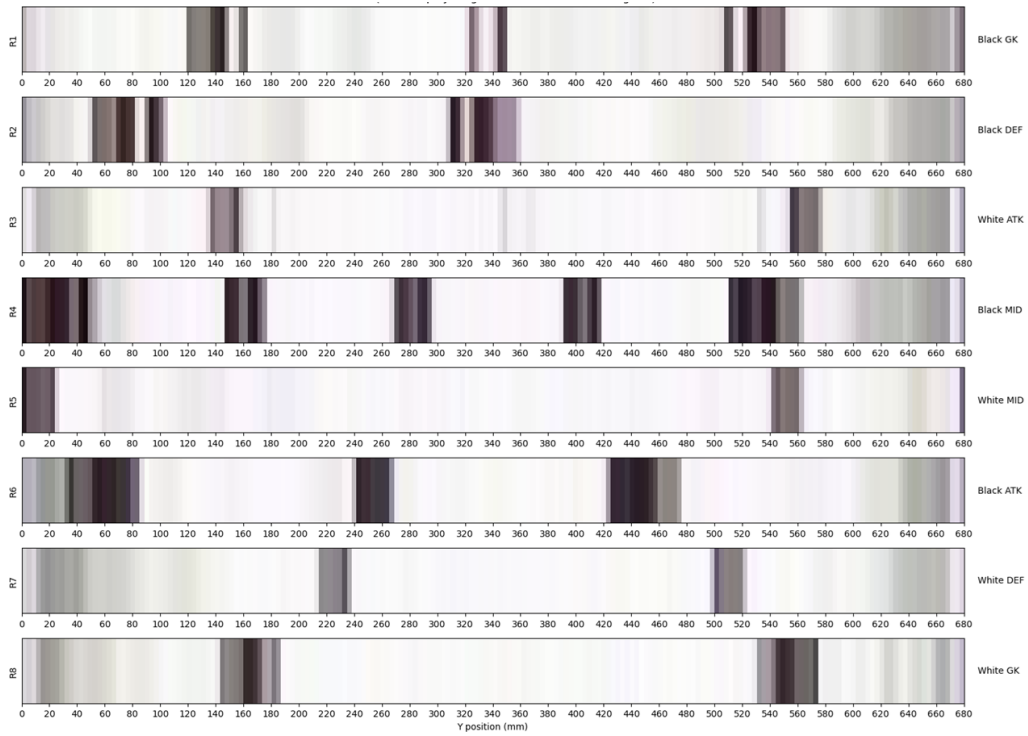


Figure 4.8: Color strips sampled along all 8 rod lines. Dark bands correspond to Black figurines and stoppers, bright bands correspond to White figurines and the rod surface.

Figure 4.9 shows the grayscale intensity signal along the same 8 rods before bilateral filtering. The signal consists of peaks and valleys. The valleys represent the Black figurines and stoppers, while the flat high-intensity regions represent the rod surface and White figurines, as both appear bright in grayscale.

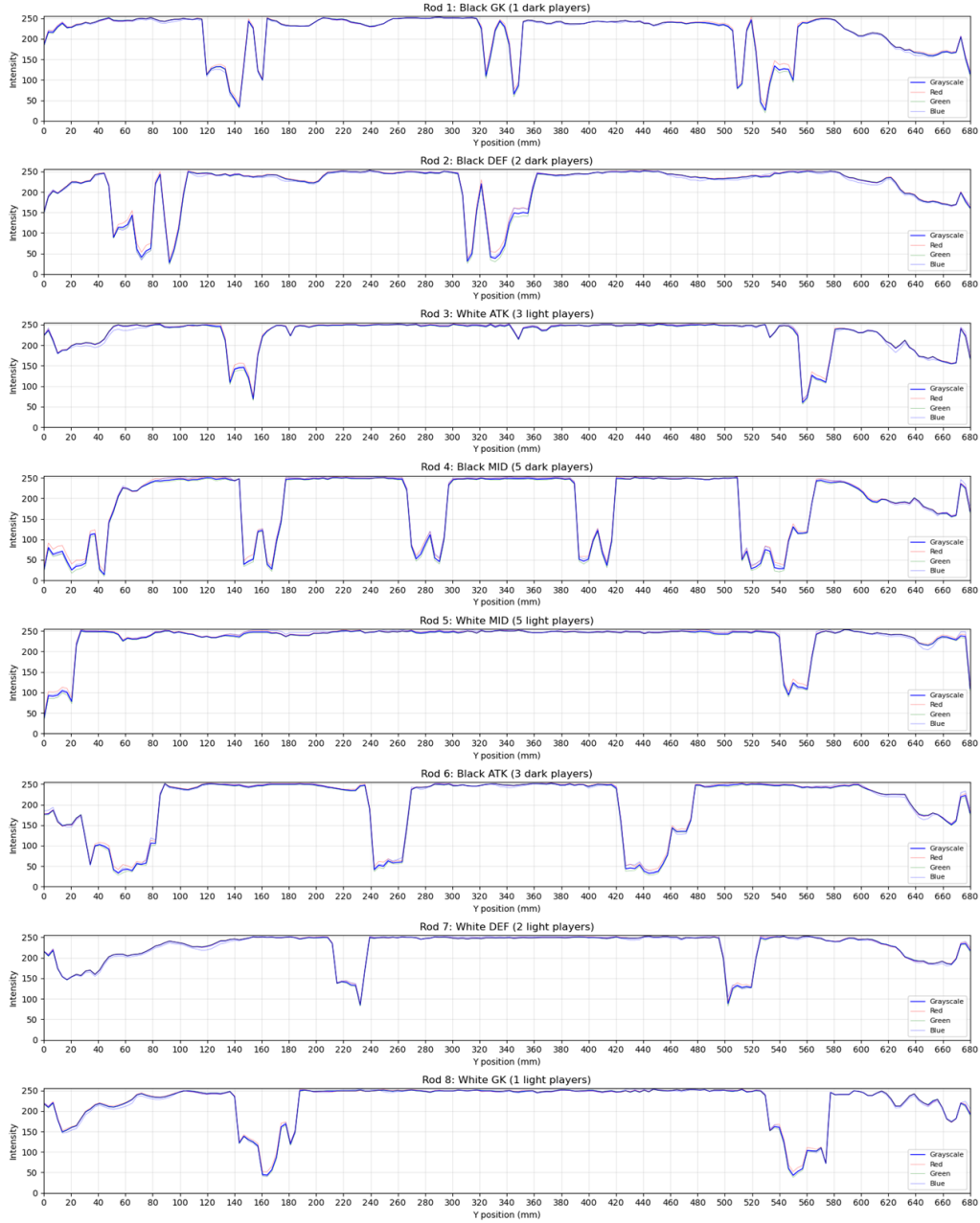


Figure 4.9: Raw grayscale intensity profiles along all 8 rod lines before bilateral filtering. Valleys correspond to dark Black figurines and stoppers. Flat high-intensity regions correspond to the rod surface and White figurines.

4.5.2 Peak Detection and Player Position

Figure 4.10 shows the inverted intensity signal after bilateral filtering for all 8 rods. The grey dashed line is the 50th percentile of the inverted signal and the orange dashed line is the adaptive threshold, which is the 50th percentile plus an offset of 30. The blue shaded

regions are the detected peaks above the threshold. The green lines with labels P1 to P5 mark the detected player positions and the red line marks the point of interest (POI). As discussed in Section 3.7, all player position offsets are computed relative to this POI. The stopper-based approach successfully identifies the correct number of players on each rod. In cases where fewer than two stopper regions are detected, the player positions for that rod are not updated and the previously accepted positions are retained by the smart stabilisation mechanism.

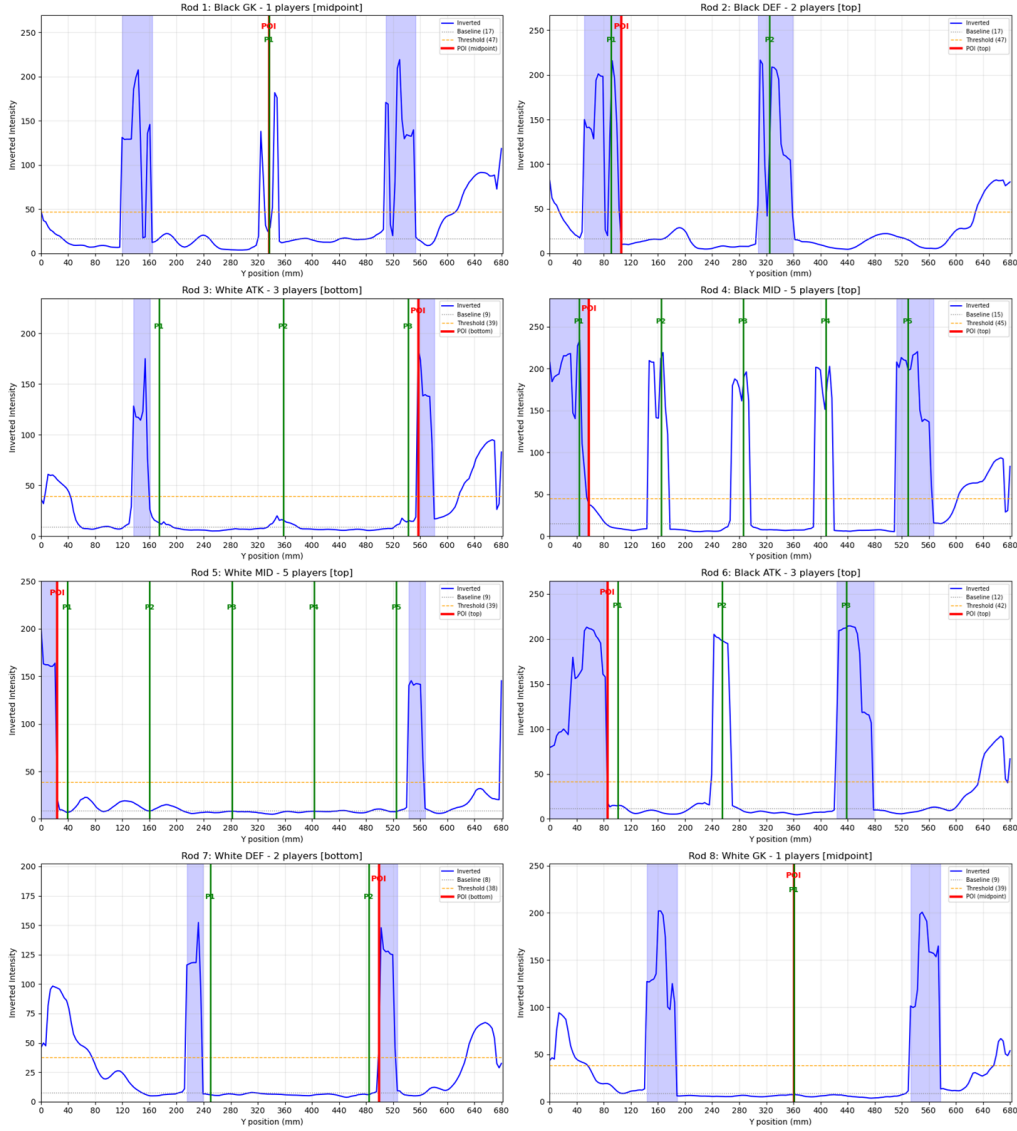


Figure 4.10: Inverted intensity profiles after bilateral filtering for all 8 rods. The grey dashed line is the 50th percentile of the inverted signal. The orange dashed line is the adaptive threshold. Blue shaded regions are detected peaks, green lines mark detected player positions, and the red line marks the point of interest used for offset computation.

4.5.3 Final Player Position Overview

Figure 4.11 summarises the full player position detection result in the warped bird’s-eye view. The red dots mark the points of interest, the green circles mark the detected player positions on the rods, and the rectangle boxes show the hit zones associated with each player as discussed in Section 3.8.

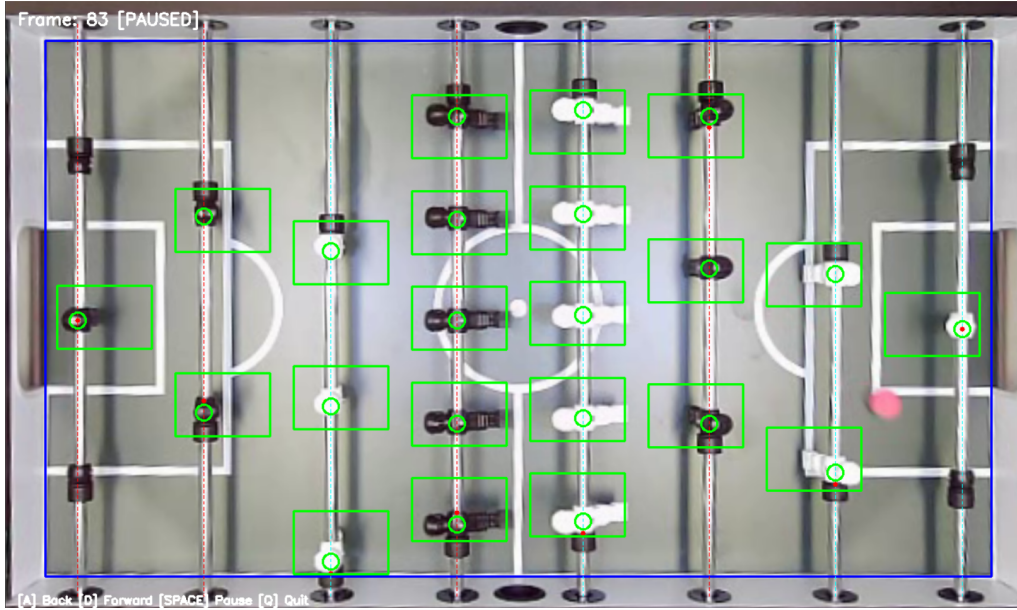


Figure 4.11: Overview of detected player positions in the warped bird’s-eye view. Red dots are the points of interest, green circles are the detected player positions, and rectangles are the corresponding hit zones.

4.6 Hit Detection Results

4.6.1 Hit Count and Type Distribution

A total of 3184 hits are detected across 41 non-Draw videos, giving an average of 77.7 hits per video. Table 4.3 shows the breakdown by hit type and Table 4.4 shows the breakdown by hit category.

Dribble is the most frequent category at 36.93%, followed by Kick at 29.37%. Together these two categories account for over 66% of all detected hits, which is physically expected as dribbling and kicking are the most common interactions in foosball. Brush at 16.02% represents deflections where the ball glanced off a figurine. Block at 4.81% is the least frequent category, which is expected as a full reversal of ball direction is a relatively rare event. The team distribution is nearly equal with White at 50.09% and Black at 49.91%, confirming that the hit detection is not biased toward either team.

Table 4.3: Hit type distribution across the full dataset.

Hit Type	Count	Percentage
Strong Dribble & Kick	782	24.56%
Strong Kick	771	24.21%
Brush	510	16.02%
Stop (No Kick)	410	12.88%
Weak Dribble & Kick	211	6.63%
Dribble (No Kick)	183	5.75%
Weak Kick	164	5.15%
Block	153	4.81%
Total	3184	100.00%

Table 4.4: Hit category distribution across the full dataset.

Hit Category	Count	Percentage
Dribble	1176	36.93%
Kick	935	29.37%
Brush	510	16.02%
Stop	410	12.88%
Block	153	4.81%
Total	3184	100.00%

Table 4.5: Hit distribution by team.

Team	Count	Percentage
White	1595	50.09%
Black	1589	49.91%
Total	3184	100.00%

4.6.2 Hit Detection Qualitative Results

Figure 4.12 shows the warped bird's-eye view during an active hit event. All 22 player positions are clearly shown as green circles along with their corresponding labels. In addition these player positions, all hit zones are shown as green rectangles. These Hit zones are projected on the field plane where the ball is actually rolling. The red rectangle highlights the active hit zone of player 3.3 where the ball is currently located. The information panel in Figure 4.13 shows a previously registered hit by player 5.4 at frame 106 which is classified as a Weak Dribble & Kick. This is the latest hit to be detected and the total hit count shows the sum of all hits in the video so far. The ball is at position $X = 611$ mm, $Y = 447$ mm with a status of "On Field".

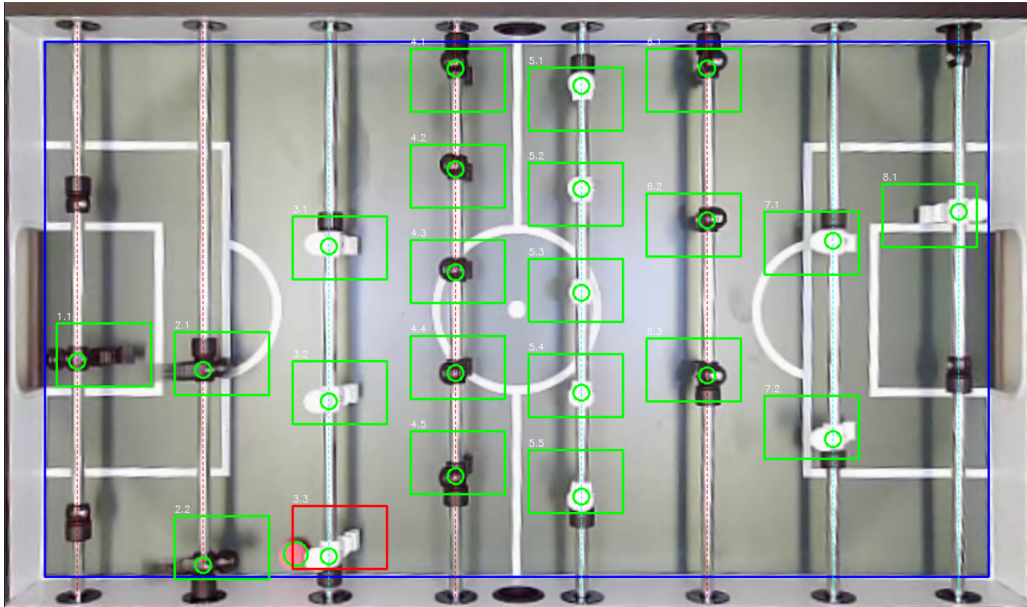


Figure 4.12: Warped bird's-eye view during an active hit event. Green circles are detected player positions, green rectangles are hit zones, and the red rectangle highlights the active hit zone of player 3.3 where the ball is currently located.

SCORE

Black: 0

White: 0

BALL TRACKING

Frame: 106

Time: 3.533s

Status: On Field

POSITION

X: 611 mm

Y: 447 mm

PHYSICS

Speed: 0.00 m/s

Accel: -12.7 m/s²

HIT DETECTION

Zone: None

Total Hits: 6

HIT!

Frame: 106

Player: 5.4

Type: Weak Dribble & Kick

CONTROLS

[SPACE] Pause/Play

[Q] Quit & Save

Figure 4.13: Real-time information panel at frame 106 showing ball tracking status, position, physics, and the detected hit event i.e. player 5.4, classified as Weak Dribble & Kick.

4.7 TCN Classification Results

4.7.1 Evaluation Protocol

Training and testing are repeated for 5 different random seeds: 42, 7, 123, 99, and 21. Both `torch.manual_seed` and `np.random.seed` are set at the start of each iteration to control weight initialisation and data shuffling respectively. The following metrics are collected per seed: overall sequence-level accuracy, per-class accuracy for White, Black, and Draw, hard vote accuracy, and soft vote accuracy. The mean, standard deviation, minimum, and maximum of all metrics are reported across all 5 seeds.

The performance of the model is compared against two baselines. The first is a uniform random baseline of 0.333, which assumes that the classifier assigns one of the three classes with equal probability. The second is a majority-class baseline of 0.375, computed on the test set. Since both White and Black classes have 18 sequences each out of 48 sequences, a classifier that always predicts either of these classes would achieve $\frac{18}{48} = 0.375$. All accuracy values reported in this section are compared against both baselines.

4.7.2 Training Curves and Convergence Behaviour

Figure 4.14 shows the training curves for seed 21. Across the 20 epochs, the training loss decreases from approximately 1.12 to 0.83. This indicates that the model is learning from the training data. The train accuracy also increases steadily from approximately 0.54 to 0.63. That said, it is also observed that the validation loss and validation accuracy are very noisy and both curves fluctuate between approximately 0.98 and 1.22 and 0.28 and 0.46 respectively.

The gap between train and validation loss and accuracy increases as the training is carried on and this indicates over-fitting. Though the model is learning, it is failing to generalise on the test data. This behavior of the model is expected for small dataset size of only 294 training sequences. The validation accuracy reaches its peak of approximately 0.46 at epoch 5, which is very early in the training and although it reaches similar peak again, it never crosses this peak which indicates early generalisation before model is over-fitted.

The `ReduceLROnPlateau` scheduler does not produce any visible effect on the validation loss as it remains noisy throughout training with no clear sudden drop at any epoch.

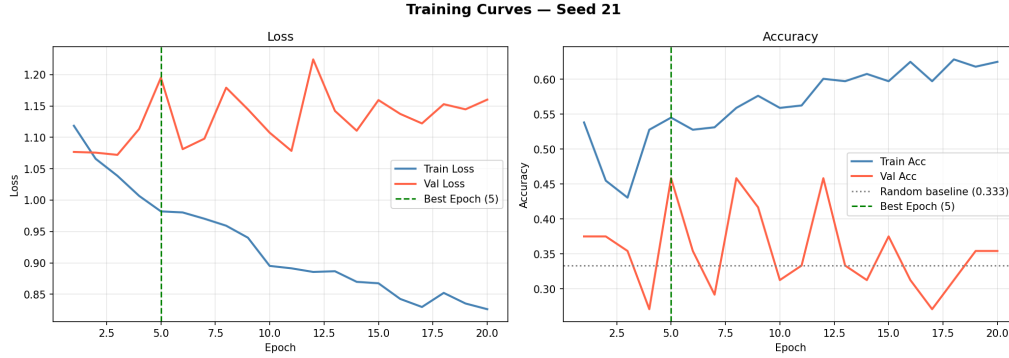


Figure 4.14: Training curves for seed 21. Left: train and validation loss per epoch. Right: train and validation accuracy per epoch. The green dashed line marks the best epoch at epoch 5. The dotted line is the random baseline of 0.333.

4.7.3 Sequence-Level Accuracy Across Seeds

Figure 4.15 shows the per-seed accuracy by class across all 5 seeds and Figure 4.16 shows the mean $\pm$ standard deviation summary across all 5 seeds. The numerical results are summarised in Table 4.6.

Table 4.6: Mean, standard deviation, minimum and maximum accuracy across 5 seeds.

Metric	Mean	Std	Min	Max
Overall	0.487	0.017	0.458	0.500
White	0.800	0.159	0.556	0.944
Black	0.256	0.027	0.222	0.278
Draw	0.367	0.215	0.167	0.667
Hard Vote	0.575	0.100	0.500	0.750
Soft Vote	0.525	0.050	0.500	0.625

The overall sequence-level accuracy is 0.487 ± 0.017 across all 5 seeds. A standard deviation of 0.017 indicates that the overall accuracy is stable and not sensitive to the choice of seed. White achieves the highest per-class accuracy of 0.800 ± 0.159 . However the high standard deviation of 0.159 indicates that White accuracy varies considerably across seeds and this is also visible in Figure 4.15 where White accuracy ranges from approximately 0.56 to 0.94. Black achieves the lowest per-class accuracy of 0.256 ± 0.027 . This is connected to the class imbalance in the dataset, where Black has only 11 videos which means the model has seen far fewer Black examples during training despite the class weights. The low standard deviation of 0.027 confirms that Black accuracy is consistently poor regardless of the seed. Draw achieves 0.367 ± 0.215 . The high standard deviation indicates that Draw prediction is highly sensitive to the random initialisation. This behaviour is understandable as Draw games have balanced possession and similar positional patterns to close wins, making

them difficult to distinguish reliably.

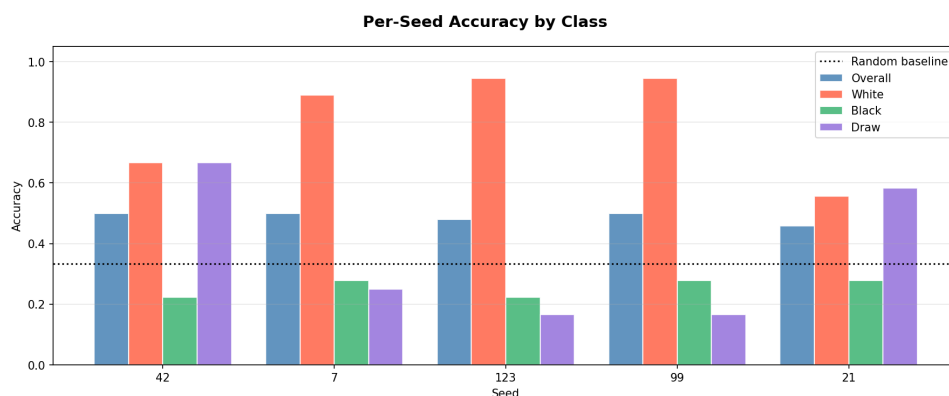


Figure 4.15: Per-seed accuracy by class across all 5 seeds. The dotted line is the random baseline of 0.333.

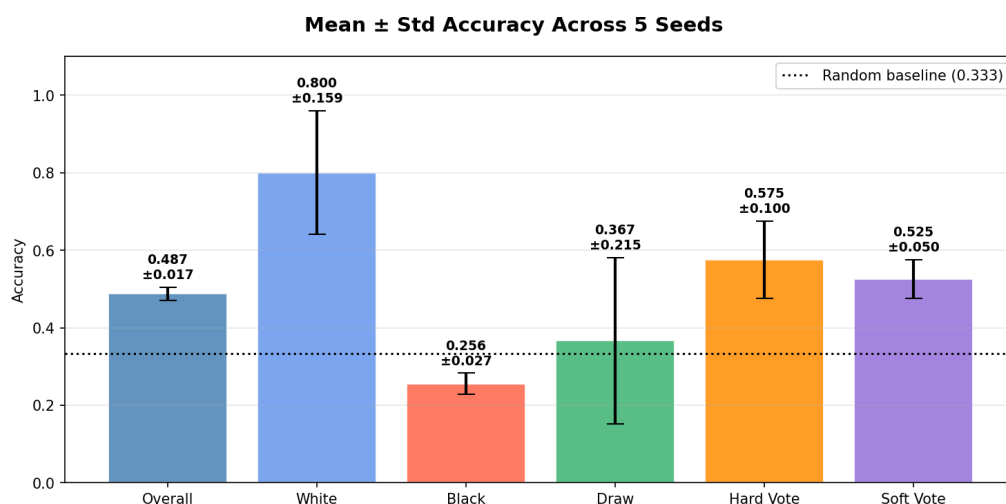


Figure 4.16: Mean $\pm$ standard deviation accuracy across all 5 seeds for all metrics. The dotted line is the random baseline of 0.333.

4.7.4 Confusion Matrix Analysis

Figure 4.17 shows the confusion matrix for seed 21 in both raw counts and row-normalised percentages. The diagonal cells represent correct predictions and the off-diagonal cells represent misclassifications.

White achieves 55.6% accuracy with 10 correct predictions out of 18. Five White sequences are misclassified as Black and three as Draw. Black achieves only 27.8% accuracy with five correct predictions out of 18. An interesting observation is that 10 out of 18 Black sequences are misclassified as White i.e. more than half of all Black sequences are predicted

as White. Draw achieves the highest accuracy of 58.3% with seven correct predictions out of 12.

The most frequent misclassification pattern is Black being predicted as White. This is consistent with the mean accuracy results and this behavior can be physically explained by the class imbalance i.e. the model has seen many more White examples during training and therefore has a bias toward predicting White. Though the class weights successfully prevented the model from completely ignoring any class, it is clear that this measure is not sufficient to completely overcome the issue because of small dataset size.

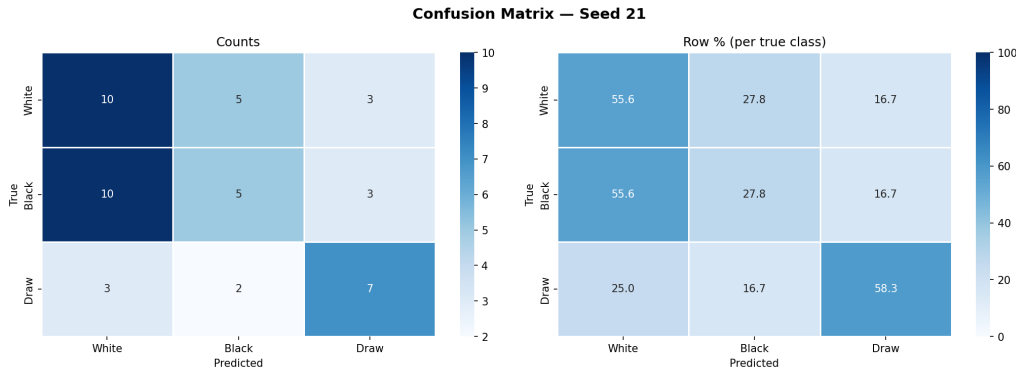


Figure 4.17: Confusion matrix for seed 21. Left: raw counts. Right: row-normalised percentages per true class.

4.7.5 Predicted Probability Distributions

Figure 4.18 shows the predicted probability distributions for seed 21. For each true class, the figure shows the distribution of predicted probabilities for all three classes across the test sequences.

For true class White (18 sequences), the $P(\text{White})$ distribution is spread across approximately 0.1–0.6 and is not concentrated near 1.0. The probabilities for all three classes are clustered in a similar range of 0.2–0.4 and this indicates that the model is uncertain about White predictions and is not confidently assigning high probability to the correct class.

For true class Black (18 sequences), $P(\text{White})$ extends from 0.1 to 0.8, indicating that many Black sequences receive high White probability. $P(\text{Black})$ is clustered near 0.1–0.2 and this indicates that the model assigns very low probability to the correct class for most Black sequences. This is consistent with the confusion matrix results.

For true class Draw (12 sequences), $P(\text{Draw})$ has distributions ranging from 0.1–0.7. The model shows some confidence in Draw predictions for certain sequences.

Overall, the probability distributions are not sharply concentrated near 1.0 for any class. All predicted probabilities are quite spread out for most sequences. This is expected given

the small dataset size and the inherent difficulty of predicting a foosball game outcome from positional features alone.

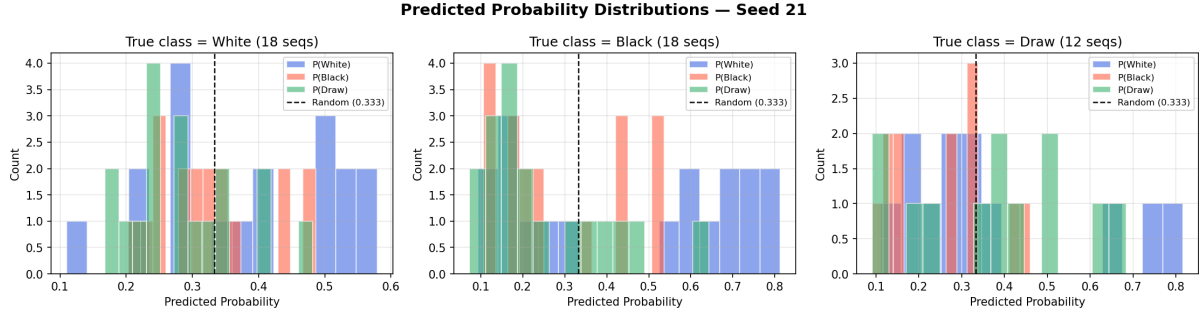


Figure 4.18: Predicted probability distributions for seed 21. Each subplot shows the distribution of predicted probabilities for all three classes across sequences belonging to one true class.

4.7.6 Video-Level Accuracy via Voting

Figure 4.19 shows the video-level voting results for seed 21 and Figure 4.20 shows the final accuracy comparison across all evaluation levels.

For seed 21, both hard vote and soft vote achieve $5/8$ correct = 0.625. The correctly predicted videos are V2 (Black), V4 (White), V5 (White), V6 (White), and V8 (Draw). The incorrectly predicted videos are V1 (predicted White, true Black), V3 (predicted White, true Black), and V7 (predicted White, true Draw). All three incorrect predictions involve the model predicting White and this confirms the White bias observed in the confusion matrix. Hard and soft vote produce identical results for this seed, suggesting that the confidence levels across the six sequences per video are consistent enough that averaging the probability vectors did not change the outcome.

From Figure 4.20, the mean sequence-level accuracy is 0.487 ± 0.017 , the mean hard vote accuracy is 0.575 ± 0.100 , and the mean soft vote accuracy is 0.525 ± 0.050 . Video-level voting improves over sequence-level accuracy for both voting strategies, with hard vote giving the largest improvement of approximately 9%. This confirms that combining the 6 sequences per video helps smooth out the mistakes. Soft vote is more stable than hard vote and this is indicated by the lower standard deviation of 0.050 compared to 0.100, suggesting that using the full probability vector is more reliable than a simple majority vote when the dataset is small and predictions are noisy. All three metrics are above the random baseline of 0.333 and the majority-class baseline of 0.375, confirming that video-level voting successfully aggregates sequence predictions into more reliable game-level decisions.

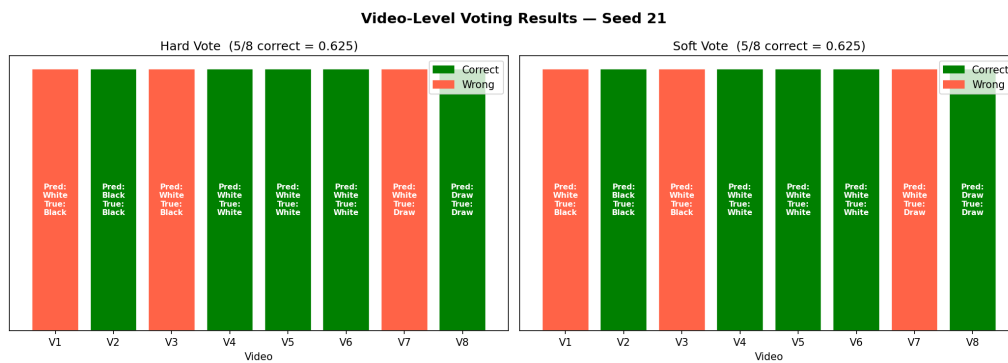


Figure 4.19: Video-level voting results for seed 21. Left: hard vote. Right: soft vote. Green bars indicate correct predictions and red bars indicate incorrect predictions.

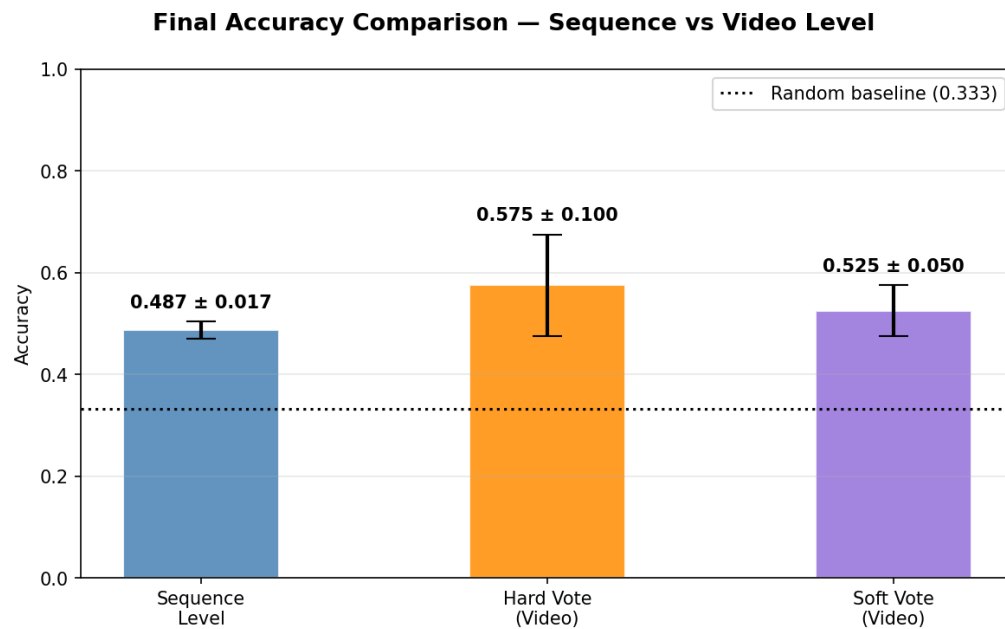


Figure 4.20: Final accuracy comparison across evaluation levels. Sequence-level accuracy: 0.487 ± 0.017 . Hard vote accuracy: 0.575 ± 0.100 . Soft vote accuracy: 0.525 ± 0.050 . The dotted line is the random baseline of 0.333.

5 Conclusion

5.1 Summary and Connection to the Introduction

This thesis presents a complete end-to-end pipeline for automated foosball game analysis using a single fixed overhead webcam. The calibration achieves an excellent RMS reprojection error of 0.1585 pixels. The ball is directly detected in 73.63% of all frames and the Kalman filter handles 4,099 occluded frames i.e. 4.00% of all frames with no gaps in the trajectory. The intensity-profile-based player position detection successfully estimates all 22 figurines' positions across eight rods in each frame. The hit detection engine detects 3184 hits across 41 non-Draw videos with Dribble and Kick being the most frequent categories. The TCN-based winner prediction achieves a mean sequence-level accuracy of 0.487 ± 0.017 , a hard-vote video-level accuracy of 0.575 ± 0.100 and a soft-vote video-level accuracy of 0.525 ± 0.050 , across five seeds, consistently above both the uniform random baseline of 0.333 and the majority-class baseline of 0.375.

The problem posed in the introduction is addressed and partially solved. A complete pipeline from raw video to winner prediction is successfully built and evaluated. However, the prediction accuracy remains moderate due to the small dataset size and class imbalance. The model consistently beats the random baseline across all five seeds, confirming that positional features extracted from a single webcam carry meaningful information about the game outcome. The six-sequence split augments the small dataset, class weighting ensures non-zero accuracy for all three classes, and video-level voting reduces individual sequence errors. The Draw class proved harder than expected as draw games resemble close wins, and the Black class fell below the random baseline due to severe class imbalance with only 11 Black win videos.

5.2 Limitations and Future Work

The primary limitation is the dataset size of 57 videos, with significant class imbalance, which causes high variance across seeds and prevents the model from generalising reliably. The system also ignores rod rotation entirely as it cannot be detected from an overhead camera without computationally expensive methods, which leave shot power and direction out of the feature set. Finally, the setup is not universal as the HSV thresholds, calibration,

and intensity profile parameters are specific to the physical table, the webcam and lighting used in this work.

Future work should focus on collecting more data with better class balance, which would allow more sophisticated models such as deeper TCNs, attention mechanisms, and transformers to be used. Rod rotation angles could potentially be estimated using methods from Hagens [4] and Horst et al. [3] and this would create eight new features, i.e. one per rod. Real-time deployment could also be explored by optimising bottlenecks such as bilateral filtering, following the real-time inference approach of Horst et al. [3].

Bibliography

- [1] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” 2018. [Online]. Available: <https://arxiv.org/abs/1803.01271>
- [2] R. Janssen, J. De Best, and R. Van De Molengraft, “Real-time ball tracking in a semi-automated foosball table,” in *Robot Soccer World Cup*. Springer, 2009, pp. 128–139.
- [3] R. Horst, D. Hagens, E. Hergenröther, and A. Weinmann, “Real time state detection of a foosball game using cnn-based computer vision,” in *Science and Information Conference*. Springer, 2024, pp. 343–354.
- [4] D. Hagens, “Real-time game state detection on a semi-automatic foosballtable,” 2023.
- [5] R. Tsai, “A versatile camera calibration technique for high-accuracy 3d machine vision metrology using off-the-shelf tv cameras and lenses,” *IEEE Journal on Robotics and Automation*, vol. 3, no. 4, pp. 323–344, 1987.
- [6] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [7] Y. Deldjoo and R. Ebrahimi Atani, “A low-cost infrared-optical head tracking solution for virtual 3d audio environment using the nintendo wii-remote,” *Entertainment Computing*, vol. 12, 01 2016.
- [8] D. C. Brown, “Close-range camera calibration,” *Photogrammetric engineering*, vol. 37, no. 8, pp. 855–866, 1971.
- [9] T. Tykkälä, “Real-time image-based rgb-d camera motion tracking and environment mapping,” Ph.D. dissertation, 11 2013.
- [10] J. J. Moré, “The levenberg-marquardt algorithm: implementation and theory,” in *Numerical analysis: proceedings of the biennial Conference held at Dundee, June 28–July 1, 1977*. Springer, 2006, pp. 105–116.
- [11] G. Bradski, “The opencv library.” *Dr. Dobb’s Journal: Software Tools for the Professional Programmer*, vol. 25, no. 11, pp. 120–123, 2000.
- [12] J. Pech-Pacheco, G. Cristobal, J. Chamorro-Martinez, and J. Fernandez-Valdivia, “Di-atom autofocus in brightfield microscopy: a comparative study,” in *Proceedings 15th International Conference on Pattern Recognition. ICPR-2000*, vol. 3, 2000, pp. 314–317 vol.3.
- [13] S. Garrido-Jurado, R. Muñoz-Salinas, F. Madrid-Cuevas, and M. Marín-Jiménez,

- “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320314000235>
- [14] M. A. Fischler and R. C. Bolles, “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, no. 6, p. 381–395, Jun. 1981. [Online]. Available: <https://doi.org/10.1145/358669.358692>
- [15] R. Sheffer and A. Wiesel, “Pnp-net: A hybrid perspective-n-point network,” *ArXiv*, vol. abs/2003.04626, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:212644661>
- [16] A. Ranganathan, “The levenberg-marquardt algorithm,” *Tutorial on LM algorithm*, vol. 11, no. 1, pp. 101–110, 2004.
- [17] R. Hartley and A. Zisserman, *Multiple view geometry in computer vision*. Cambridge university press, 2003.
- [18] A. R. Smith, “Color gamut transform pairs,” in *Proceedings of the 5th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH ’78. New York, NY, USA: Association for Computing Machinery, 1978, p. 12–19. [Online]. Available: <https://doi.org/10.1145/800248.807361>
- [19] A. Tatomir and A. Färber, “Saturation determination for multiphase systems in porous medium using light transmission method,” 05 2007.
- [20] R. M. Haralick, S. R. Sternberg, and X. Zhuang, “Image analysis using mathematical morphology,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-9, no. 4, pp. 532–550, 1987.
- [21] I. H. Ali, “Digital images edge detection using mathematical morphology operations hanaa,” 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:67821352>
- [22] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 03 1960. [Online]. Available: <https://doi.org/10.1115/1.3662552>
- [23] G. Welch, G. Bishop *et al.*, “An introduction to the kalman filter,” 1995.
- [24] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *2016 IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468.
- [25] C. Tomasi and R. Manduchi, “Bilateral filtering for gray and color images,” in *Sixth International Conference on Computer Vision (IEEE Cat. No.98CH36271)*, 1998, pp. 839–846.
- [26] A. Graves, *Supervised Sequence Labelling*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 5–13. [Online]. Available: https://doi.org/10.1007/978-3-642-24797-2_2
- [27] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, “Backpropagation applied to handwritten zip code recognition,” *Neural computation*,

- vol. 1, no. 4, pp. 541–551, 1989.
- [28] F. Yu and V. Koltun, “Multi-scale context aggregation by dilated convolutions,” *arXiv preprint arXiv:1511.07122*, 2015.
 - [29] A. Van Den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, K. Kavukcuoglu *et al.*, “Wavenet: A generative model for raw audio,” *arXiv preprint arXiv:1609.03499*, vol. 12, no. 1, 2016.
 - [30] A. Enkhbat, M. Gochoo, T. Shih, P. Cheewaparakobkit, W. Aditya, T. Duy Quy, H. Lin, and Y.-T. Lin, “Using hybrid models for action correction in instrument learning based on ai,” *IEEE Access*, vol. PP, pp. 1–1, 01 2024.
 - [31] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
 - [32] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
 - [33] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, F. Bach and D. Blei, Eds., vol. 37. Lille, France: PMLR, 07–09 Jul 2015, pp. 448–456. [Online]. Available: <https://proceedings.mlr.press/v37/ioffe15.html>
 - [34] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.
 - [35] I. Marin, A. Skelin, and T. Grujic, “Empirical evaluation of the effect of optimization and regularization techniques on the generalization performance of deep convolutional neural network,” *Applied Sciences*, vol. 10, p. 7817, 11 2020.
 - [36] M. Lin, Q. Chen, and S. Yan, “Network in network,” 2014. [Online]. Available: <https://arxiv.org/abs/1312.4400>
 - [37] Y. Guo, G. Ggg, J. Wang, H. Yu, and R.-C. Chen, “Real-time facial affective computing on mobile devices,” *Sensors*, vol. 20, p. 870, 02 2020.
 - [38] H. He and E. A. Garcia, “Learning from imbalanced data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
 - [39] Z.-H. Zhou and X.-Y. Liu, “Training cost-sensitive neural networks with methods addressing the class imbalance problem,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 18, no. 1, pp. 63–77, 2006.
 - [40] A. Manconi, G. Armano, M. Gnocchi, and L. Milanesi, “A soft-voting ensemble classifier for detecting patients affected by covid-19,” *Applied Sciences*, vol. 12, p. 7554, 07 2022.